

In[104]:=

```
(* ===== *)
(* Script 0: CondorNet A-Matrix Stability & Gershgorin Report *)
(* ===== *)

ClearAll[];

Print["[1/10] Ensuring A-matrix files are public..."];

SetPermissions[
  "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices/Epoch1_A_Matrix.csv",
  "Public"];
SetPermissions[
  "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices/Epoch2_A_Matrix.csv",
  "Public"];
SetPermissions[
  "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices/Epoch3_A_Matrix.csv",
  "Public"];
SetPermissions[
  "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices/Epoch4_A_Matrix.csv",
  "Public"];
SetPermissions[
  "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices/Epoch5_A_Matrix.csv",
  "Public"];

Print["[2/10] Defining base URL and file list..."];

base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;

Print["[3/10] Defining safe import and analysis functions..."];
```

```

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw],
    raw,
    Print["[WARN] Non-matrix content at: ", url];
    $Failed
  ]
];

analyzeA[matrix_] := Module[
{
  A = matrix,
  eigs, eigRe, rhoA, expA, eigExp, rhoExp, gersh, dt = 1.0,
  diag, rowSums, radii
},

eigs = Eigenvalues[A];
eigRe = Re[eigs];
rhoA = Max[Abs[eigs]];
expA = MatrixExp[A*dt];
eigExp = Eigenvalues[expA];
rhoExp = Max[Abs[eigExp]];

diag = Diagonal[A];
rowSums = Total[Abs[#]] & /@ A;
radii = rowSums - Abs[diag];
gersh = Max[diag + radii];

<|
  "Matrix" → A,
  "Eigenvalues" → eigs,
  "MaxReEigenvalue" → Max[eigRe],
  "MinReEigenvalue" → Min[eigRe],
  "SpectralRadiusA" → rhoA,
  "SpectralRadiusExpA" → rhoExp,
  "GershgorinBound" → gersh,
  "GershCenters" → diag,
  "GershRadii" → radii,
  "ContinuousStableQ" → (Max[eigRe] < 0),
  "DiscreteStableQ" → (rhoExp < 1)
|>
];

Print["[4/10] Importing and analyzing A-matrices..."];

```

```

rawResults = analyzeA[safeImport[#]] & /@ urls;
results = Select[rawResults, # != $Failed &];
validFiles = Pick[files, rawResults != $Failed];

comparison = Dataset[
  AssociationThread[validFiles,
    KeyDrop[#, {"Matrix", "GershCenters", "GershRadii"}] & /@ results]
];

epochNums = ToExpression@StringReplace[
  validFiles,
  "Epoch" ~~ x_ ~~ "_A_Matrix.csv" -> x
];

maxRe = results[[All, "MaxReEigenvalue]];
rhoExp = results[[All, "SpectralRadiusExpA]];
gersh = results[[All, "GershgorinBound]];

norm[x_] := (x - Min[x]) / (Max[x] - Min[x] + 10^-12);

Print["[5/10] Building stability drift plots..."];

normalizedDriftPlot = ListLinePlot[
{
  Transpose[{epochNums, norm[maxRe]}],
  Transpose[{epochNums, norm[rhoExp]}]
},
PlotLegends -> {"MaxReEigenvalue (normalized)", "SpectralRadiusExpA (normalized)"},
Frame -> True,
FrameLabel -> {"Epoch", "Normalized Value"},
PlotTheme -> "Scientific",
GridLines -> Automatic,
ImageSize -> Large,
PlotLabel -> "Normalized Stability Drift Across Epochs"
];

microMaxRePlot = ListLinePlot[
  Transpose[{epochNums, maxRe}],
  PlotStyle -> Red,
  Frame -> True,

```

```

FrameLabel → {"Epoch", "MaxReEigenvalue"},
PlotRange → {All, {Min[maxRe], Max[maxRe]}},
GridLines → Automatic,
ImageSize → Large,
PlotLabel → "MaxReEigenvalue (Micro-Scale)"
];

microRhoExpPlot = ListLinePlot[
  Transpose[{epochNums, rhoExp}],
  PlotStyle → Blue,
  Frame → True,
  FrameLabel → {"Epoch", "SpectralRadiusExpA"},
  PlotRange → {All, {Min[rhoExp], Max[rhoExp]}},
  GridLines → Automatic,
  ImageSize → Large,
  PlotLabel → "SpectralRadiusExpA (Micro-Scale)"
];

deltaMaxRe = maxRe - First[maxRe];
deltaRhoExp = rhoExp - First[rhoExp];

deltaPlot = ListLinePlot[
  {
    Transpose[{epochNums, deltaMaxRe}],
    Transpose[{epochNums, deltaRhoExp}]
  },
  PlotLegends → {"Δ MaxReEigenvalue", "Δ SpectralRadiusExpA"},
  Frame → True,
  FrameLabel → {"Epoch", "Deviation from Epoch 1"},
  PlotTheme → "Scientific",
  GridLines → Automatic,
  ImageSize → Large,
  PlotLabel → "Deviation from Baseline Stability"
];

Print["[6/10] Building eigenvalue and Gershgorin plots..."];

eigenScatterPlots =
Table[
  With[{eigs = results[[k, "Eigenvalues"]]},
    ListPlot[

```

```

Transpose[{Re[eigs], Im[eigs]}],
PlotStyle → Directive[Blue, PointSize[0.012]],
Frame → True,
FrameLabel → {"Re( $\lambda$ )", "Im( $\lambda$ )"},
PlotRange → All,
AspectRatio → 1,
GridLines → Automatic,
ImageSize → Large,
PlotLabel → "Eigenvalue Spectrum - " <> validFiles[[k]]
]
],
{k, Length[validFiles]}
];

gershDiscPlots =
Table[
Module[{A = results[[k, "Matrix"]],
diag, rowSums, radii, discs},
diag = Diagonal[A];
rowSums = Total[Abs[ $\#$ ]] & /@ A;
radii = rowSums - Abs[diag];
discs = Table[
Circle[{Re[diag[[i]]], Im[diag[[i]]]}, radii[[i]]],
{i, Length[diag]}
];
Show[
Graphics[{LightGray, discs}],
Frame → True,
FrameLabel → {"Re(z)", "Im(z)"},
AspectRatio → 1,
PlotRange → All,
GridLines → Automatic,
ImageSize → Large,
PlotLabel → "Gershgorin Discs - " <> validFiles[[k]]
]
],
{k, Length[validFiles]}
];

Print["[7/10] Building 3D Gershgorin drift diagnostics..."];

```

```

gershTriples[res_] := Module[
  {c = res["GershCenters"], r = res["GershRadii"]},
  Transpose[{Re[c], Im[c], r}]
];

gershAll = gershTriples /@ results;
baseGersh = First[gershAll];

rawGersh3DData = Flatten[
  Table[
    With[{epoch = epochNums[[k]], gk = gershAll[[k]]},
      Table[
        {epoch, gk[[i, 1]], gk[[i, 3]]},
        {i, Length[gk]}
      ]
    ],
    {k, Length[epochNums]}
  ],
  1
];

rawGersh3DPlot = ListPointPlot3D[
  rawGersh3DData,
  AxesLabel → {"Epoch", "Re(center)", "Radius"},
  PlotTheme → "Scientific",
  BoxRatios → {2, 1, 1},
  ImageSize → Large,
  PlotLabel → "Gershgorin Discs Across Epochs (Re(center), Radius)"
];

deltaGersh3DData = Flatten[
  Table[
    With[{epoch = epochNums[[k]], gk = gershAll[[k]], gb = baseGersh},
      Table[
        {
          epoch,
          gk[[i, 1]] - gb[[i, 1]],
          gk[[i, 3]] - gb[[i, 3]]
        },
        {i, Min[Length[gk], Length[gb]]}
      ]
    ]
  ]

```

```

    ]
  ],
  {k, Length[epochNums]}
],
1
];

deltaGersh3DPlot = ListPointPlot3D[
  deltaGersh3DDData,
  AxesLabel → {"Epoch", "ΔRe(center)", "ΔRadius"},
  PlotTheme → "Scientific",
  BoxRatios → {2, 1, 1},
  ImageSize → Large,
  PlotLabel → "Δ Gershgorin Discs vs Epoch (Re(center), Radius)"
];

deltaGersh3DImagData = Flatten[
  Table[
    With[{epoch = epochNums[[k]], gk = gershAll[[k]], gb = baseGersh},
      Table[
        {
          epoch,
          gk[[i, 2]] - gb[[i, 2]],
          gk[[i, 3]] - gb[[i, 3]]
        },
        {i, Min[Length[gk], Length[gb]]}
      ]
    ],
    {k, Length[epochNums]}
  ],
  1
];

deltaGersh3DImagPlot = ListPointPlot3D[
  deltaGersh3DImagData,
  AxesLabel → {"Epoch", "ΔIm(center)", "ΔRadius"},
  PlotTheme → "Scientific",
  BoxRatios → {2, 1, 1},
  ImageSize → Large,
  PlotLabel → "Δ Gershgorin Discs vs Epoch (Im(center), Radius)"
];

```

```

];

Print["[8/10] Assembling notebook report..."];

nbCells = Join[
{
  Cell["CondorNet A-Matrix Stability Report", "Title"],
  Cell["Generated automatically from all available epoch A-matrices.", "Text"],
  Cell["Global Stability Summary", "Section"],
  Cell[BoxData@ToBoxes[comparison], "Output"],
  Cell["High-Resolution Drift Diagnostics", "Section"],
  Cell[BoxData@ToBoxes[normalizedDriftPlot], "Output"],
  Cell[BoxData@ToBoxes[microMaxRePlot], "Output"],
  Cell[BoxData@ToBoxes[microRhoExpPlot], "Output"],
  Cell[BoxData@ToBoxes[deltaPlot], "Output"],
  Cell["Eigenvalue Spectra by Epoch", "Section"]
},
Flatten[
  Table[
    {
      Cell["", "Subsection"],
      Cell[BoxData@ToBoxes[eigenScatterPlots[[i]], "Output"]
    },
    {i, Length[eigenScatterPlots]}
  ],
  1
],
{
  Cell["Gershgorin Discs by Epoch", "Section"]
},
Flatten[
  Table[
    {
      Cell["", "Subsection"],
      Cell[BoxData@ToBoxes[gershDiscPlots[[i]], "Output"]
    },
    {i, Length[gershDiscPlots]}
  ],
  1
]

```



```

],
{
  Cell["Gershgorin Drift Diagnostics (3D)", "Section"],
  Cell[BoxData@ToBoxes[rawGersh3DPlot], "Output"],
  Cell[BoxData@ToBoxes[deltaGersh3DPlot], "Output"],
  Cell[BoxData@ToBoxes[deltaGersh3DImagPlot], "Output"]
}
];

```

```
nb = NotebookPut[Notebook[nbCells]];
```

```
Print["[9/10] Notebook report created."];
```

```
(* Optional: local PDF export of the report *)
```

```
reportFile = ;
```

```
Export[reportFile, nb];
```

```
If[FileExistsQ[reportFile],
```

```
  Print["[10/10] PDF report exported: ", reportFile],
```

```
  Print["[10/10] PDF export FAILED"]
```

```
];
```

```
nb
```

```
[1/10] Ensuring A-matrix files are public...
```

```
[2/10] Defining base URL and file list...
```

```
[3/10] Defining safe import and analysis functions...
```

```
[4/10] Importing and analyzing A-matrices...
```

```
[5/10] Building stability drift plots...
```

```
[6/10] Building eigenvalue and Gershgorin plots...
```

```
[7/10] Building 3D Gershgorin drift diagnostics...
```

```
[8/10] Assembling notebook report...
```

```
[9/10] Notebook report created.
```

```
[10/10] PDF report exported: CondorNet_A_Matrix_Stability_Report.pdf
```

```
Out[155]=
```

```
NotebookObject[["M-d42fa81b-69d7-48e1-b4e5-b28b0d39521f"]]
```

```
In[156]:=
```

```
(* ===== *)
```

```
(* Script 0.5: 3D Brainimation of CondorNet A-Matrix Evolution *)
```

```
(* ===== *)

ClearAll[];

Print["[1/12] Loading A-matrices..."];

base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];

Adata = Select[safeImport /@ urls, # != $Failed &];

Print["[2/12] Computing spectral geometry..."];

computeGeometry[A_] := Module[
  {vals, vecs, radii, pts, spheres, fibers},

  vals = Eigenvalues[A];
  vecs = Eigenvectors[A];

  radii = Total[Abs[A], {2}] - Abs[Diagonal[A]];
  pts = Transpose[{Re[vals], Im[vals], radii}];

  spheres = Table[
    {pts[[i], radii[[i]]},
    {i, Length[pts]}
  ];
];
```

```

fibers = Table[
  Line[{pts[[i]], pts[[i]] + 0.2 Normalize[Re[vecs[[i, 1 ;; 3]]]}],
  {i, Length[vals]}
];

<|"Points" → pts, "Spheres" → spheres, "Fibers" → fibers|>
];

geom = computeGeometry /@ Adata;

Print["[3/12] Building 3D frames..."];

frames = Table[
  With[{g = geom[[i]]},
    Graphics3D[
      {
        Opacity[0.25], ColorData["Rainbow"][[0.2],
        Table[
          Sphere[g["Spheres"][[k, 1]], g["Spheres"][[k, 2]],
          {k, Length[g["Spheres"]]}
        ],
      },
      ColorData["Rainbow"][[0.8],
      PointSize[0.015], Point[g["Points"]],
      Thick, ColorData["Rainbow"][[0.5], g["Fibers"]
    ],
    Boxed → False, Lighting → "Neutral",
    PlotRange → All, ImageSize → Large
  ]
],
  {i, Length[geom]}
];

Print["[4/12] Reversing frames for growth effect..."];
revFrames = Reverse[frames];

Print["[5/12] Slowing down animation..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[6/12] Rasterizing frames..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

```

```

Print["[7/12] Exporting MP4 locally..."];

localFile = "brainimation_temp.mp4";
Export[
  localFile,
  cleanFrames,
  "VideoEncoding" → "H264",
  "FrameRate" → 12
];

If[FileExistsQ[localFile],
  Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[];
];

Print["[8/12] Uploading to Wolfram Cloud (primary method)..."];

cloudPath = "jerryamyluv/VideoExports/Brainimation_A_Matrix_Evolution.mp4";

cloudObj = CloudExport[
  cleanFrames,
  "MP4",
  cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[9/12] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"]; Print["URL: ", cloudObj],

  Print["✗ Upload failed – trying fallback method..."];

  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/Brainimation_A_Matrix_Evolution_v2.mp4"
  ];
  SetPermissions[cloudObj, "Public"];

```

```

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Fallback upload successful!"]; Print["URL: ", cloudObj],
  Print["✗ Both methods failed. Local file saved at: ", localFile];
];
];

Print["[10/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share URL: ", shareURL];

Print["[11/12] Displaying preview frame..."];
preview = First[frames];
preview

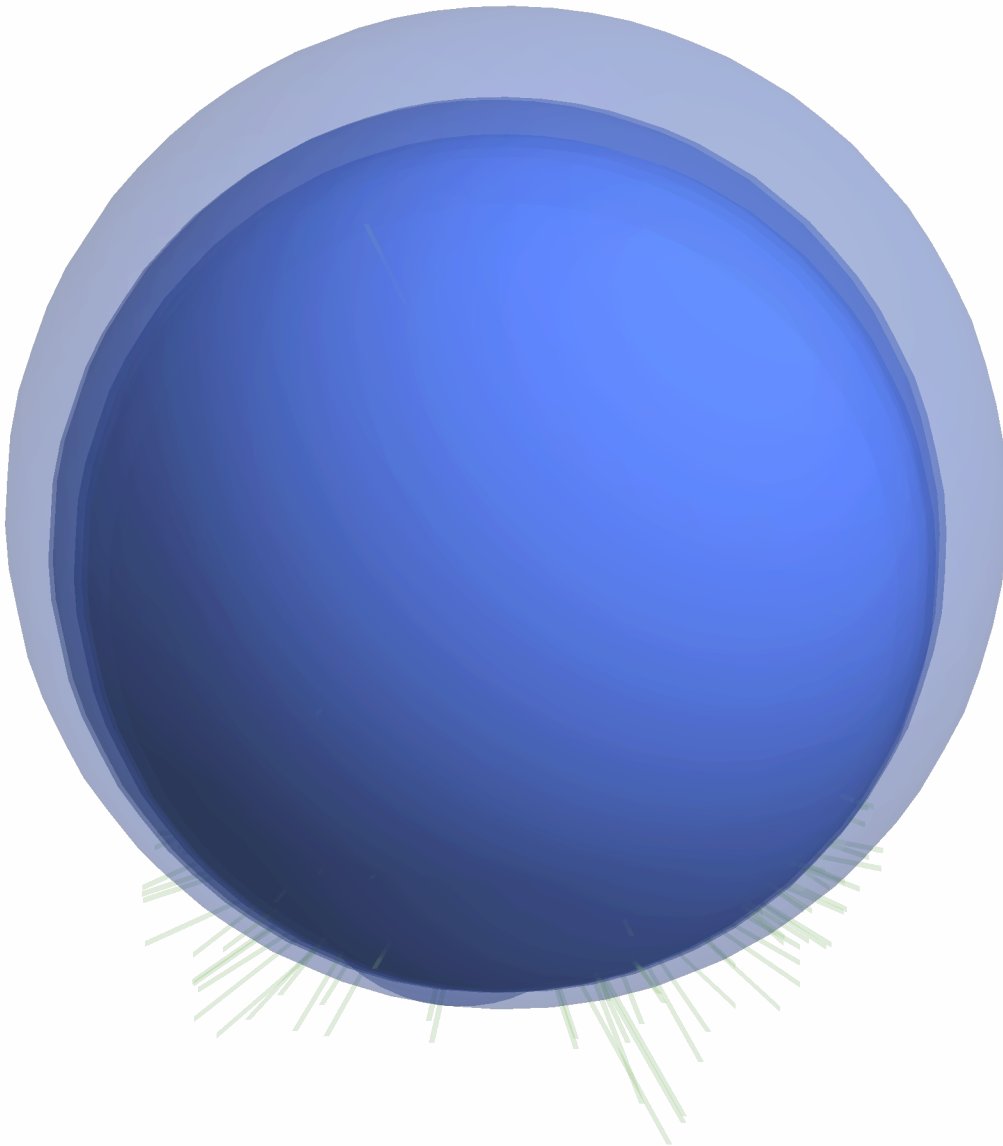
Print["[12/12] Script 0.5 complete! ✓"];
Print["Video location: ", cloudObj];
Print["Direct link: ", shareURL];

[1/12] Loading A-matrices...
[2/12] Computing spectral geometry...
[3/12] Building 3D frames...
[4/12] Reversing frames for growth effect...
[5/12] Slowing down animation...
[6/12] Rasterizing frames...
[7/12] Exporting MP4 locally...
✓ Local export successful
[8/12] Uploading to Wolfram Cloud (primary method)...
[9/12] Verifying upload...
✓ Upload successful!
URL: CloudObject[
  https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/Brainimation_A]
  _Matrix_Evolution.mp4
[10/12] Getting shareable URL...

```

```
Share URL: CloudObject[  
  https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/Brainimation_A[  
    _Matrix_Evolution.mp4  
  HTTPSPath]  
[11/12] Displaying preview frame...
```

Out[189]=



[12/12] Script 0.5 complete! ✓

Video location: CloudObject[

https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/Brainimation_A_Matrix_Evolution.mp4

Direct link: CloudObject[

https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/Brainimation_A_Matrix_Evolution.mp4

HTTPSPath]

In[193]:=

```
(* ===== *)
(* Script 0.6: 3D Eigenvector Flow Field Brainimation *)
(* ===== *)
```

ClearAll[];

Print["[1/12] Loading A-matrices..."];

base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";

files = {

"Epoch1_A_Matrix.csv",

"Epoch2_A_Matrix.csv",

"Epoch3_A_Matrix.csv",

"Epoch4_A_Matrix.csv",

"Epoch5_A_Matrix.csv"

};

urls = base <> # & /@ files;

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},

If[MatrixQ[raw], raw, \$Failed]

];

Adata = Select[safeImport /@ urls, # != \$Failed &];

Print["[2/12] Computing eigenvector flow geometry..."];

computeFlow[A_] := Module[

{vals, vecs, pts, arrows, mags},

vals = Eigenvalues[A];

```

vecs = Eigenvectors[A];

pts = Transpose[{Re[vals], Im[vals], Table[0, Length[vals]]}];

mags = Norm /@ vecs;
arrows = Table[
  Arrow[{pts[[i]], pts[[i]] + 0.3 Normalize[Re[vecs[[i, 1 ;; 3]]]}],
  {i, Length[vals]}
];

<|"Points" → pts, "Arrows" → arrows, "Magnitudes" → mags|>
];

flow = computeFlow /@ Adata;

minMag = Min[Flatten[flow[[All, "Magnitudes"]]]];
maxMag = Max[Flatten[flow[[All, "Magnitudes"]]]];
neonColor[v_] := ColorData["NeonColors"][Rescale[v, {minMag, maxMag}]];

Print["[3/12] Building 3D frames..."];

frames = Table[
  With[{f = flow[[i]]},
    Graphics3D[
      {
        PointSize[0.02],
        Table[{neonColor[f["Magnitudes"][[k]]], Point[f["Points"][[k]]},
          {k, Length[f["Points"]]}],
        Thick,
        Table[{neonColor[f["Magnitudes"][[k]]], f["Arrows"][[k]]},
          {k, Length[f["Arrows"]]}],
      },
      Background → RGBColor[0.02, 0.02, 0.05],
      Boxed → False,
      Lighting → "Neutral",
      PlotRange → All,
      ImageSize → 700
    ]
  ],
  {i, Length[flow]}
];

```



```

Print["[4/12] Reversing frames..."];
revFrames = Reverse[frames];

Print["[5/12] Slowing down animation..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[6/12] Rasterizing..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[7/12] Exporting MP4 locally..."];

localFile = "eigenvector_flow_temp.mp4";
Export[localFile, cleanFrames, "VideoEncoding" → "H264", "FrameRate" → 12];

If[FileExistsQ[localFile],
  Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[];
];

Print["[8/12] Uploading to Wolfram Cloud..."];

cloudPath = "jerryamyluv/VideoExports/EigenvectorFlow_Brainimation.mp4";

cloudObj = CloudExport[
  cleanFrames, "MP4", cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[9/12] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"]; Print["URL: ", cloudObj],

  Print["✗ Upload failed – trying fallback..."];
  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/EigenvectorFlow_Brainimation_v2.mp4"
  ]
];

```

```

];
SetPermissions[cloudObj, "Public"];
];

Print["[10/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share URL: ", shareURL];

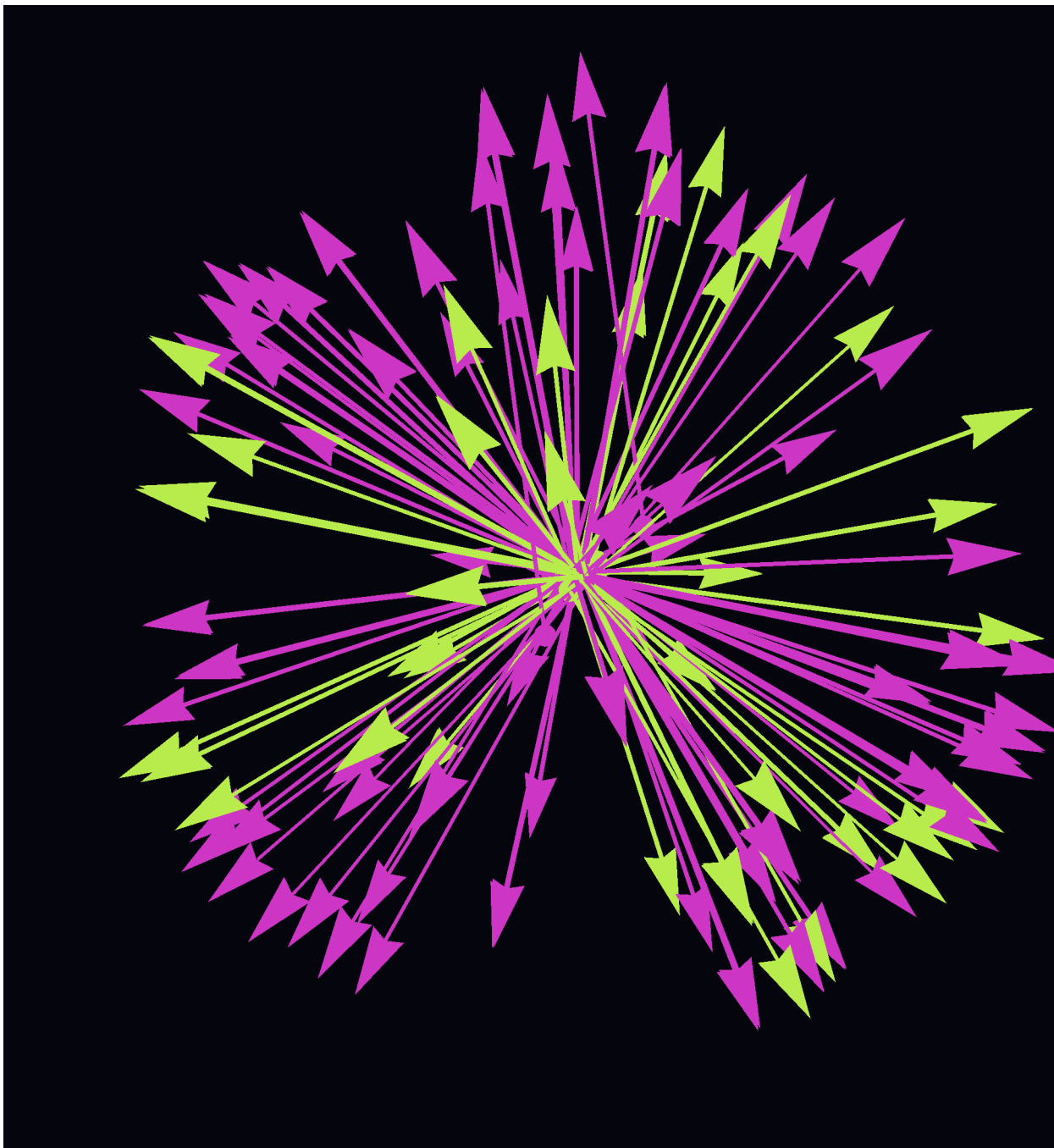
Print["[11/12] Preview frame..."];
preview = First[frames]; preview

Print["[12/12] Script 0.6 complete! ✓"];

[1/12] Loading A-matrices...
[2/12] Computing eigenvector flow geometry...
[3/12] Building 3D frames...
[4/12] Reversing frames...
[5/12] Slowing down animation...
[6/12] Rasterizing...
[7/12] Exporting MP4 locally...
✓ Local export successful
[8/12] Uploading to Wolfram Cloud...
[9/12] Verifying upload...
✓ Upload successful!
URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/]
      EigenvectorFlow_Brainimation.mp4
[10/12] Getting shareable URL...
Share URL: CloudObject[
      https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/][HTTPSPath]
      EigenvectorFlow_Brainimation.mp4
[11/12] Preview frame...

```

Out[228]=



[12/12] Script 0.6 complete! ✓

In[230]:=

```
(* ===== *)
(* Script 0.7: 3D A-Matrix Tensor-Field Brainimation *)
(* ===== *)

ClearAll[];
```

```

Print["[1/12] Loading A-matrices..."];

base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];
Adata = Select[safeImport /@ urls, # != $Failed &];

Print["[2/12] Computing tensor field geometry..."];

computeTensor[A_] := Module[
  {rows, pts, vecs, arrows},

  rows = A[[All, 1 ;; 3]];
  pts = Table[{i, 0, 0}, {i, Length[rows]}];
  vecs = Normalize /@ rows;

  arrows = Table[
    Arrow[{pts[[i]], pts[[i]] + 0.3 vecs[[i]]},
    {i, Length[rows]}
  ];

  <|"Points" → pts, "Arrows" → arrows, "Magnitudes" → Norm /@ rows>
];

tensor = computeTensor /@ Adata;

minMag = Min[Flatten[tensor[[All, "Magnitudes"]]]];
maxMag = Max[Flatten[tensor[[All, "Magnitudes"]]]];

```

```

neonColor[v_] := ColorData["NeonColors"][Rescale[v, {minMag, maxMag}]];

Print["[3/12] Building 3D frames..."];

frames = Table[
  With[{t = tensor[[i]]},
    Graphics3D[
      {
        PointSize[0.02],
        Table[{neonColor[t["Magnitudes"][[k]], Point[t["Points"][[k]]},
          {k, Length[t["Points"]]}],
        Thick,
        Table[{neonColor[t["Magnitudes"][[k]], t["Arrows"][[k]]},
          {k, Length[t["Arrows"]]}],
      },
      Background → RGBColor[0.02, 0.02, 0.05],
      Boxed → False,
      Lighting → "Neutral",
      PlotRange → All,
      ImageSize → 700
    ]
  ],
  {i, Length[tensor]}
];

Print["[4/12] Reversing frames..."];
revFrames = Reverse[frames];

Print["[5/12] Slowing down animation..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[6/12] Rasterizing..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[7/12] Exporting MP4 locally..."];

localFile = "tensor_field_temp.mp4";
Export[localFile, cleanFrames, "VideoEncoding" → "H264", "FrameRate" → 12];

If[FileExistsQ[localFile],

```

```

Print["✓ Local export successful"],
Print["✗ Local export FAILED"]; Abort[];
];

Print["[8/12] Uploading to Wolfram Cloud..."];

cloudPath = "jerryamyluv/VideoExports/TensorField_Brainimation.mp4";

cloudObj = CloudExport[
  cleanFrames, "MP4", cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[9/12] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"]; Print["URL: ", cloudObj],

  Print["✗ Upload failed – trying fallback..."];
  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/TensorField_Brainimation_v2.mp4"
  ];
  SetPermissions[cloudObj, "Public"];
];

Print["[10/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share URL: ", shareURL];

Print["[11/12] Preview frame..."];
preview = First[frames]; preview

Print["[12/12] Script 0.7 complete! ✓"];

[1/12] Loading A-matrices...
[2/12] Computing tensor field geometry...
[3/12] Building 3D frames...

```

```

[4/12] Reversing frames...
[5/12] Slowing down animation...
[6/12] Rasterizing...
[7/12] Exporting MP4 locally...
✓ Local export successful
[8/12] Uploading to Wolfram Cloud...
[9/12] Verifying upload...
✓ Upload successful!
URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/
      TensorField_Brainimation.mp4
[10/12] Getting shareable URL...
Share URL: CloudObject[
      https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/][HTTPSPath]
      TensorField_Brainimation.mp4
[11/12] Preview frame...

```

Out[265]=



[12/12] Script 0.7 complete! ✓

In[267]:=

```
(* ===== *)  
(* Script 0.8: CondorNet Neural Organ Composite Brainimation *)  
(* ===== *)  
  
ClearAll[];
```



```
Print["[1/12] Loading A-matrices..."];
```

```
base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;
```

```
safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];
Adata = Select[safeImport /@ urls, # != $Failed &];
```

```
Print["[2/12] Computing composite geometry..."];
```

```
computeComposite[A_] := Module[
  {vals, vecs, diag, rowSums, radii, pts, spheres, flowArrows, rho},

  vals = Eigenvalues[A];
  vecs = Eigenvectors[A];

  diag = Diagonal[A];
  rowSums = Total[Abs[#]] & /@ A;
  radii = rowSums - Abs[diag];

  pts = Transpose[{Re[vals], Im[vals], radii}];
  spheres = Table[{pts[[i]], radii[[i]], {i, Length[pts]}}];

  flowArrows = Table[
    Arrow[{pts[[i]], pts[[i]] + 0.2 Normalize[Re[vecs[[i, 1 ;; 3]]]}],
    {i, Length[vals]}
  ];

  rho = Max[Abs[vals]];
```

```
<|"Points" → pts, "Spheres" → spheres, "Flow" → flowArrows, "Rho" → rho|>
];
```

```
comp = computeComposite /@ Adata;
```

```
minRho = Min[comp[[All, "Rho"]]];
maxRho = Max[comp[[All, "Rho"]]];
neonColor[v_] := ColorData["NeonColors"][Rescale[v, {minRho, maxRho}]]];
```

```
Print["[3/12] Building composite 3D frames..."];
```

```
frames = Table[
  With[{c = comp[[i]]},
    Graphics3D[
      {
        Opacity[0.2],
        Table[{neonColor[c["Rho"]], Sphere[c["Spheres"][[k, 1]], c["Spheres"][[k, 2]]},
          {k, Length[c["Spheres"]]}],

        PointSize[0.02],
        Table[{neonColor[c["Rho"]], Point[c["Points"]][[k]]},
          {k, Length[c["Points"]]}],

        Thick,
        Table[{neonColor[c["Rho"]], c["Flow"][[k]]},
          {k, Length[c["Flow"]]}],
      },
      Background → RGBColor[0.02, 0.02, 0.05],
      Boxed → False,
      Lighting → "Neutral",
      PlotRange → All,
      ImageSize → 700
    ]
  ],
  {i, Length[comp]}
];
```

```
Print["[4/12] Reversing frames..."];
```

```
revFrames = Reverse[frames];
```

```

Print["[5/12] Slowing down animation..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[6/12] Rasterizing..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[7/12] Exporting MP4 locally..."];

localFile = "neural_organ_temp.mp4";
Export[localFile, cleanFrames, "VideoEncoding" → "H264", "FrameRate" → 12];

If[FileExistsQ[localFile],
  Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[];
];

Print["[8/12] Uploading to Wolfram Cloud..."];

cloudPath = "jerryamyluv/VideoExports/NeuralOrgan_Brainimation.mp4";

cloudObj = CloudExport[
  cleanFrames, "MP4", cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[9/12] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"]; Print["URL: ", cloudObj],

  Print["✗ Upload failed – trying fallback..."];
  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/NeuralOrgan_Brainimation_v2.mp4"
  ];
  SetPermissions[cloudObj, "Public"];
];

```

```

Print["[10/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share URL: ", shareURL];

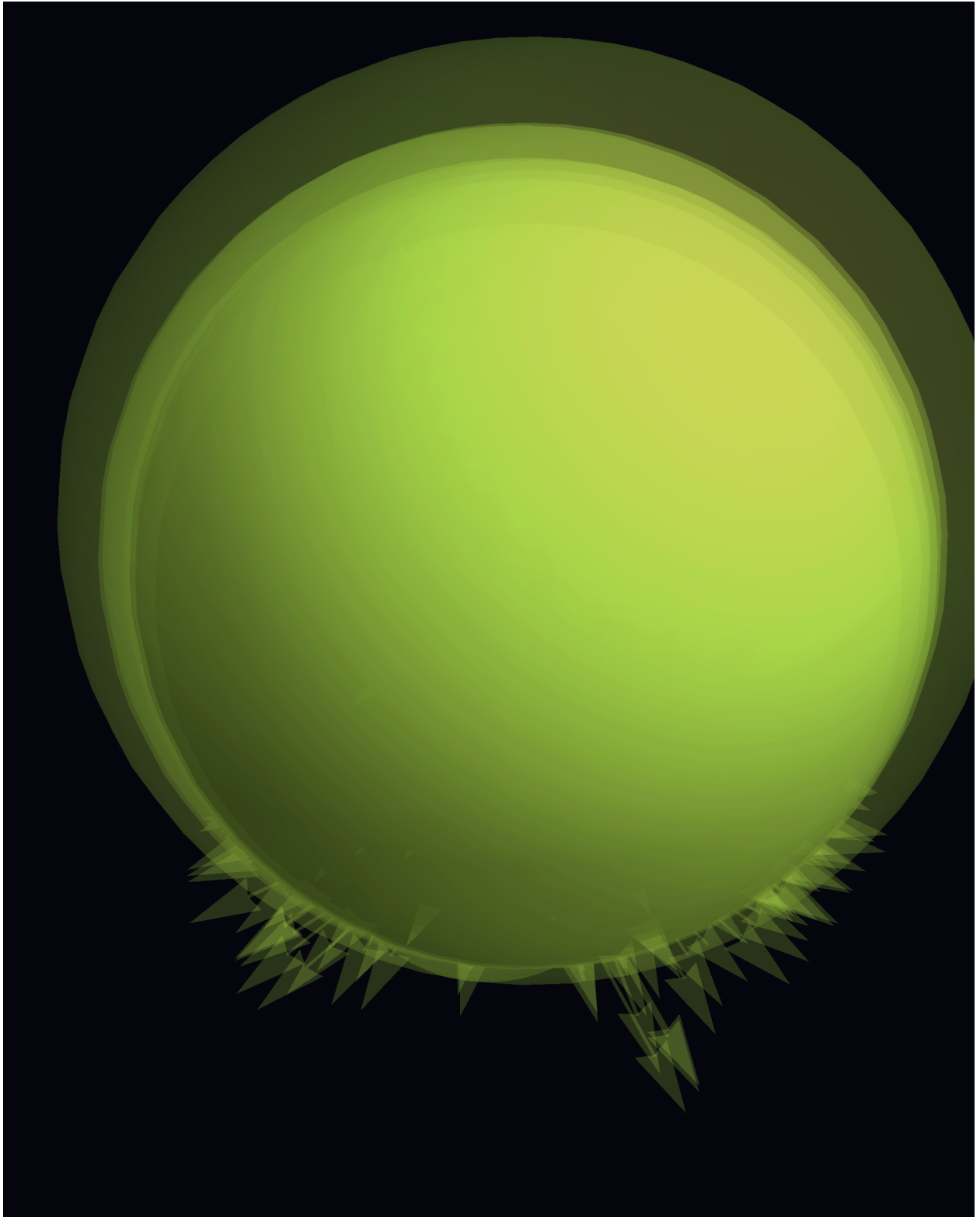
Print["[11/12] Preview frame..."];
preview = First[frames]; preview

Print["[12/12] Script 0.8 complete! ✓"];

[1/12] Loading A-matrices...
[2/12] Computing composite geometry...
[3/12] Building composite 3D frames...
[4/12] Reversing frames...
[5/12] Slowing down animation...
[6/12] Rasterizing...
[7/12] Exporting MP4 locally...
✓ Local export successful
[8/12] Uploading to Wolfram Cloud...
[9/12] Verifying upload...
✓ Upload successful!
URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/]
      NeuralOrgan_Brainanimation.mp4
[10/12] Getting shareable URL...
Share URL: CloudObject[
      https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/][HTTPSPath]
      NeuralOrgan_Brainanimation.mp4
[11/12] Preview frame...

```

Out[302]=



[12/12] Script 0.8 complete! ✓

In[304]:=

(* ===== *)

```
(* Script 0.9: CondorNet Neural Genesis Composite Brainimation *)
(* - Unified brain formation animation *)
(* ===== *)
```

```
ClearAll[];
```

```
Print["[1/14] Loading A-matrices..."];
```

```
base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;
```

```
safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];
```

```
Adata = Select[safeImport /@ urls, # != $Failed &];
```

```
If[Adata == {},
  Print["x No valid A-matrices loaded. Aborting."]; Abort[];
];
```

```
Print["[2/14] Computing global stability metrics..."];
```

```
analyzeA[matrix_] := Module[
  {A = matrix, eigs, eigRe, rhoA, diag, rowSums, radii, gersh},
  eigs = Eigenvalues[A];
  eigRe = Re[eigs];
  rhoA = Max[Abs[eigs]];
  diag = Diagonal[A];
  rowSums = Total[Abs[#] & /@ A;
  radii = rowSums - Abs[diag];
  gersh = Max[diag + radii];
```

```

<|
  "Matrix" → A,
  "Eigenvalues" → eigs,
  "Eigenvectors" → Eigenvectors[A],
  "Diag" → diag,
  "Radii" → radii,
  "SpectralRadius" → rhoA,
  "GershgorinBound" → gersh
|>
];

results = analyzeA /@ Adata;

epochNums = Range[Length[results]];
rhoAll = results[[All, "SpectralRadius"]];
gershAll = results[[All, "GershgorinBound"]];

minRho = Min[rhoAll]; maxRho = Max[rhoAll];
minG = Min[gershAll]; maxG = Max[gershAll];

Print["[3/14] Defining color maps..."];

neonEigen[v_] := ColorData["NeonColors"][Rescale[v, {minRho, maxRho}]];
neonShell[v_] := ColorData["NeonColors"][Rescale[v, {minG, maxG}]];
neonFiber[v_] := ColorData["NeonColors"][Rescale[v, {0, 1}]];
neonTensor[v_] := ColorData["NeonColors"][Rescale[v, {0, 1}]];
neonTube[v_] := ColorData["NeonColors"][Rescale[v, {minRho, maxRho}]];

Print["[4/14] Building per-epoch composite geometry..."];

computeComposite[res_, epoch_] := Module[
{
  A, vals, vecs, diag, radii, pts, spheres,
  fibers, tensorRows, tensorArrows, rho, gersh
},

  A = res["Matrix"];
  vals = res["Eigenvalues"];
  vecs = res["Eigenvectors"];
  diag = res["Diag"];
  radii = res["Radii"];
  rho = res["SpectralRadius"];
  gersh = res["GershgorinBound"];

```

```

(* Eigenvalue constellation points: use radius as z *)
pts = Transpose[{Re[vals], Im[vals], radii}];

(* Gershgorin shells as spheres *)
spheres = Table[
  {pts[[i]], radii[[i]],
   {i, Min[Length[pts], Length[radii]]}
];

(* Eigenvector fibers from eigenvalue points *)
fibers = Table[
  With[{dir = Re[vecs[[i, 1 ;; 3]]],
    Arrow[{pts[[i]], pts[[i]] + 0.25 Normalize[dir]}]
  ],
  {i, Min[Length[vals], Length[vecs]]}
];

(* Tensor field from first 3 columns of A *)
tensorRows = A[[All, 1 ;; 3]];
tensorArrows = Table[
  With[{base = {i, -1.5, 0}, dir = Normalize[tensorRows[[i]]],
    Arrow[{base, base + 0.3 dir}]
  ],
  {i, Min[Length[tensorRows], 40]} (* cap for visual clarity *)
];

<|
  "Points" → pts,
  "Spheres" → spheres,
  "Fibers" → fibers,
  "TensorArrows" → tensorArrows,
  "Rho" → rho,
  "Gersh" → gersh,
  "Epoch" → epoch
|>
];

composites = MapThread[computeComposite, {results, epochNums}];

Print["[5/14] Building stability tubes (global)..."];

```



```

stabilityTubes[epoch_, rho_, gersh_] := Module[
{
  e = epoch,
  t1, t2
},
t1 =
  Tube[{e, 2.0, 0.0}, {e + 0.1, 2.0 + 0.2 Rescale[rho, {minRho, maxRho}], 0.0}], 0.05];
t2 = Tube[{e, 2.0, -0.5}, {e + 0.1, 2.0 + 0.2 Rescale[gersh, {minG, maxG}], -0.5}], 0.05];
{t1, t2}
];

tubesPerEpoch = Map[
  stabilityTubes[{"Epoch"}, {"Rho"}, {"Gersh"}] &,
  composites
];

Print["[6/14] Constructing composite 3D frames..."];

frames = Table[
  Module[
    {
      c = composites[[i]],
      tubes = tubesPerEpoch[[i]],
      pts, spheres, fibers, tensorArrows, rho
    },
    pts = c["Points"];
    spheres = c["Spheres"];
    fibers = c["Fibers"];
    tensorArrows = c["TensorArrows"];
    rho = c["Rho"];

    Graphics3D[
      {
        (* Gershgorin shells: translucent tissue *)
        Table[
          {
            Opacity[0.15],
            neonShell[spheres[[k, 2]],
              Sphere[spheres[[k, 1]], spheres[[k, 2]]
            ],
          },
        ],
      }
    ]
  ]
];

```

```

    {k, Length[spheres]}
  ],

  (* Eigenvalue nodes: neural skeleton *)
  Table[
    {
      Opacity[0.35],
      neonEigen[rho],
      PointSize[0.02],
      Point[pts[[k]]]
    },
    {k, Length[pts]}
  ],

  (* Eigenvector fibers: axonal fibers *)
  Table[
    {
      Opacity[0.4],
      neonFiber[0.7],
      Thick,
      fibers[[k]]
    },
    {k, Length[fibers]}
  ],

  (* Tensor field: deep structural tension *)
  Table[
    {
      Opacity[0.2],
      neonTensor[0.3],
      tensorArrows[[k]]
    },
    {k, Length[tensorArrows]}
  ],

  (* Stability tubes: white-matter tracts *)
  {
    Opacity[0.25],
    neonTube[rho],
    tubes[[1]],
    tubes[[2]]
  }

```

```

    },
    Background → RGBColor[0.02, 0.02, 0.05],
    Boxed → False,
    Lighting → "Neutral",
    PlotRange → All,
    ImageSize → 700
  ]
],
{i, Length[composites]}
];

Print["[7/14] Reversing frames for outward growth..."];
revFrames = Reverse[frames];

Print["[8/14] Slowing down animation (duplicating frames)..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[9/14] Rasterizing frames for neon stability..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[10/14] Exporting MP4 locally (stable method)..."];

localFile = "neural_genesis_composite_temp.mp4";
Export[
  localFile,
  cleanFrames,
  "VideoEncoding" → "H264",
  "FrameRate" → 12
];

If[FileExistsQ[localFile],
  Print["✓ Local export successful: ", localFile],
  Print["✗ Local export FAILED"]; Abort[]
];

Print["[11/14] Uploading MP4 to Wolfram Cloud (primary: CloudExport)..."];

cloudPath = "jerryamyluv/VideoExports/NeuralGenesis_Composite_Brainimation.mp4";

cloudObj = CloudExport[
  cleanFrames,

```

```

    "MP4",
    cloudPath,
    "FrameRate" → 12,
    Permissions → "Public"
  ];

Print["[12/14] Verifying cloud upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"];
  Print["Cloud object: ", cloudObj],

  Print["✗ Upload failed – trying fallback CloudPut from local file..."];
  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/NeuralGenesis_Composite_Brainimation_v2.mp4"
  ];
  SetPermissions[cloudObj, "Public"];

  If[
    CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
    Print["✓ Fallback upload successful!"];
    Print["Cloud object: ", cloudObj],
    Print["✗ Both upload methods failed. Manual upload required."];
    Print["Local file remains at: ", localFile];
  ];
];

Print["[13/14] Getting shareable HTTPS URL..."];

shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share this URL: ", shareURL];

Print["[14/14] Displaying preview frame..."];
preview = First[frames];
preview

Print["Neural Genesis Composite Brainimation complete! ✓"];
Print["Video location: ", cloudObj];
Print["Direct link: ", shareURL];

```

```

[1/14] Loading A-matrices...
[2/14] Computing global stability metrics...
[3/14] Defining color maps...
[4/14] Building per-epoch composite geometry...
[5/14] Building stability tubes (global)...
[6/14] Constructing composite 3D frames...
[7/14] Reversing frames for outward growth...
[8/14] Slowing down animation (duplicating frames)...
[9/14] Rasterizing frames for neon stability...
[10/14] Exporting MP4 locally (stable method)...
✓ Local export successful: neural_genesis_composite_temp.mp4
[11/14] Uploading MP4 to Wolfram Cloud (primary: CloudExport)...
[12/14] Verifying cloud upload...
✓ Upload successful!
Cloud object:
  CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/]
    NeuralGenesis_Composite_Brainimation.mp4
[13/14] Getting shareable HTTPS URL...
Share this URL: CloudObject[
  https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/][HTTPSPath]
    NeuralGenesis_Composite_Brainimation.mp4
[14/14] Displaying preview frame...

```

Out[355]=



Neural Genesis Composite Brainimation complete! ✓

Video location:

CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/NeuralGenesis_Composite_Brainimation.mp4]

Direct link: CloudObject[

https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/NeuralGenesis_Composite_Brainimation.mp4][HTTPSPath]

In[148]:=

```
(* ===== *)
(* Script 1: Eigenvalue Cloud Brainimation (Cyber-Neon Edition) *)
(* Final Version: Preview + Status Messages + Stable Upload *)
(* ===== *)
```

```
ClearAll[];
```

```
Print["[1/12] Loading A-matrices..."];
```

```
(* ---- 0. Load A-matrices ---- *)
base = ;
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];
```

```
Adata = Select[safeImport /@ urls, # != $Failed &];
```

```
Print["[2/12] Computing eigenvalues..."];
```

```
(* ---- 1. Compute eigenvalue clouds ---- *)
eigvals = Eigenvalues /@ Adata;
```

```

(* Compute global magnitude range for Rescale *)
allMagnitudes = Abs[Flatten[eigvals]];
minMag = Min[allMagnitudes];
maxMag = Max[allMagnitudes];

(* Cyber-neon color function *)
neonColor[z_] := ColorData[Rescale[Abs[z], {minMag, maxMag}]];

Print["[3/12] Building cyber-neon frames..."];

(* ---- 2. Build cyber-neon frames ---- *)
frames = Table[
  Module[{vals = eigvals[[i]],
    Graphics[
      {
        Black,
        PointSize[0.02],
        Table[
          {neonColor[vals[[k]],
            Point[{Re[vals[[k]], Im[vals[[k]]]}],
            {k, Length[vals]}
          ]
        ],
      },
      Background → RGBColor[0.02, 0.02, 0.05],
      PlotRange → {{-2, 2}, {-2, 2}},
      ImageSize → 600
    ]
  ],
  {i, Length[eigvals]}
];

Print["[4/12] Reversing frames for outward growth..."];

(* ---- 3. Reverse for growth ---- *)
revFrames = Reverse[frames];

Print["[5/12] Slowing down animation (duplicating frames)..."];

(* ---- 4. Slow down the animation ---- *)
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

```

```

Print["[6/12] Rasterizing frames for neon glow stability..."];

(* ---- 5. Rasterize for neon glow stability ---- *)
cleanFrames = Rasterize[#, ] & /@ slowFrames;

Print["[7/12] Exporting MP4 locally (stable method)..."];

(* ---- 6. Export locally ---- *)
Export[
  ,
  cleanFrames,
  → ,
  → 12
];

(* Verify local file exists *)
If[FileExistsQ[],
  Print[],
  Print[],
  Abort[];
];

Print["[8/12] Uploading MP4 to cloud folder (stable method)..."];

cloud = CloudObject[
  "https://www.wolframcloud.com/obj/jerryamyluv/VideoExports/
  EigenvalueCloud_Brainanimation.mp4"
];

(* ---- 7. Upload using stable method ---- *)
Export[cloud, Import[], ];

Print["[9/12] Verifying cloud upload..."];

(* ---- 8. Verify upload ---- *)
If[
  CloudObjectInformation[cloud] != $Failed,
  Print[],
  Print[];

  retryCloud = CloudObject[

```



```

];

Export[retryCloud, Import[],];

If[
  CloudObjectInformation[retryCloud] != $Failed,
  Print[],
  Print[];
];

Print["[10/12] Setting public permissions..."];

(* ---- 9. Make public ---- *)
SetPermissions[cloud,];

Print["[11/12] Displaying preview frame..."];

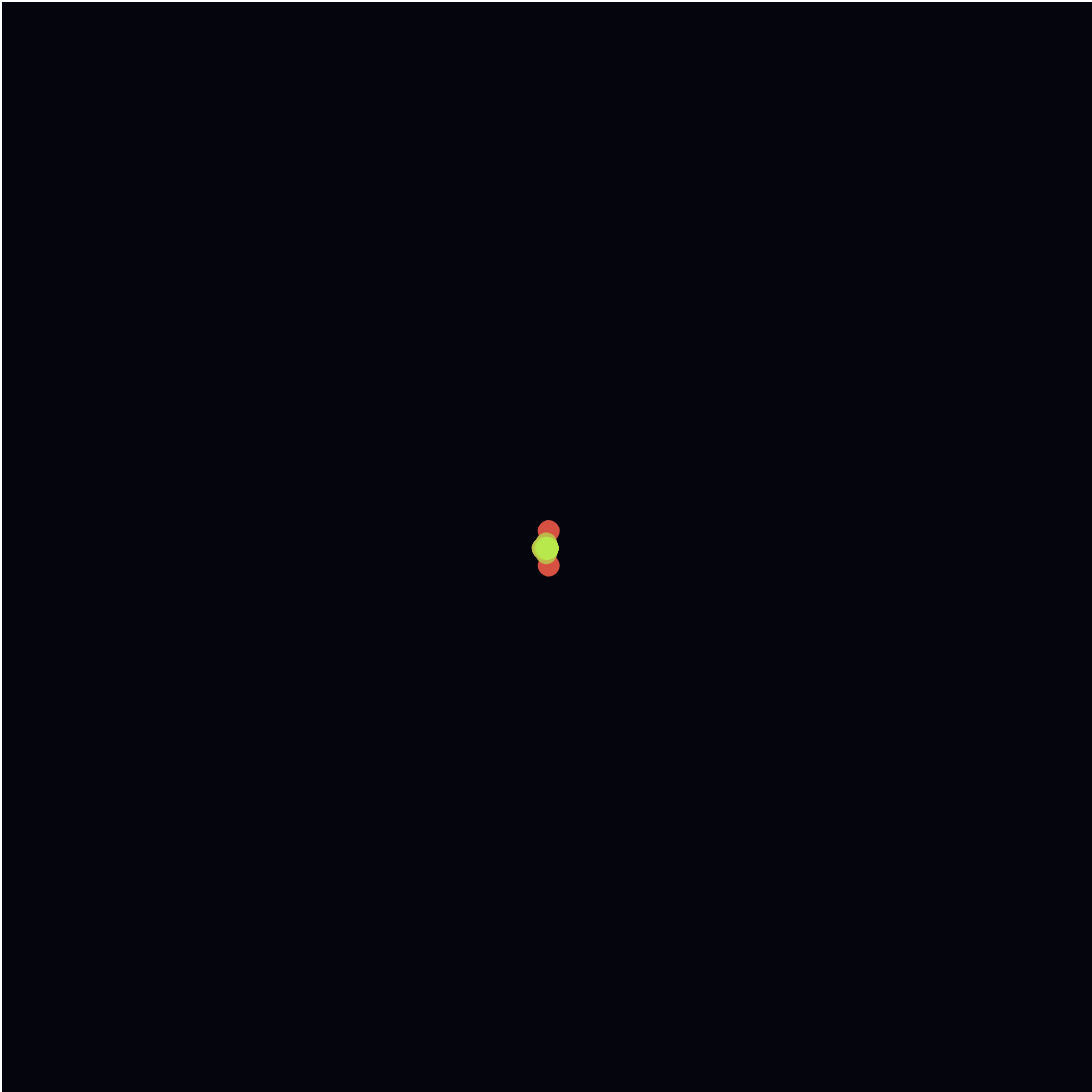
(* ---- 10. Show preview frame ---- *)
preview = First[frames];
preview

Print["[12/12] Script 1 complete."];

[1/12] Loading A-matrices...
[2/12] Computing eigenvalues...
[3/12] Building cyber-neon frames...
[4/12] Reversing frames for outward growth...
[5/12] Slowing down animation (duplicating frames)...
[6/12] Rasterizing frames for neon glow stability...
[7/12] Exporting MP4 locally (stable method)...
Local export OK
[8/12] Uploading MP4 to cloud folder (stable method)...
ExportString: Video is not a recognized ExportString format.
[9/12] Verifying cloud upload...
Upload OK
[10/12] Setting public permissions...
[11/12] Displaying preview frame...

```

Out[182]=



[12/12] Script 1 complete.

In[345]:=

```
(* ===== *)
(* Script 2: Gershgorin Disc Brainimation (Cyber-Neon Edition) *)
(* FIXED: Correct CloudObject upload syntax *)
(* ===== *)

ClearAll[];
Print["[1/12] Loading A-matrices..."];

(* ---- 0. Load A-matrices ---- *)
base = ;
files = {
```

```

    "Epoch1_A_Matrix.csv",
    "Epoch2_A_Matrix.csv",
    "Epoch3_A_Matrix.csv",
    "Epoch4_A_Matrix.csv",
    "Epoch5_A_Matrix.csv"
  };
  urls = base <> # & /@ files;

  safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
    If[MatrixQ[raw], raw, $Failed]
  ];

  Adata = Select[safeImport /@ urls, # != $Failed &];

  Print["[2/12] Computing Gershgorin discs..."];

  (* ---- 1. Compute Gershgorin discs ---- *)
  gershgorinDiscs[m_] := Module[{diag, radii},
    diag = Diagonal[m];
    radii = Total[Abs[m], {2}] - Abs[diag];
    Transpose[{diag, radii}]
  ];

  discData = gershgorinDiscs /@ Adata;

  (* Compute global radius range for neon scaling *)
  allRadii = Flatten[discData[[All, All, 2]]];
  minR = Min[allRadii];
  maxR = Max[allRadii];

  (* Cyber-neon color function *)
  neonColor[r_] := ColorData[Rescale[r, {minR, maxR}]];

  Print["[3/12] Building cyber-neon frames..."];

  (* ---- 2. Build frames ---- *)
  frames = Table[
    Module[{centers = discData[[i, All, 1]], radii = discData[[i, All, 2]]},
      Graphics[
        Table[
          {
            neonColor[radii[[k]]],

```

```

    Thick,
    Circle[{Re[centers[[k]], Im[centers[[k]]], radii[[k]]
    },
    {k, Length[centers]}
  ],
  Background → RGBColor[0.02, 0.02, 0.05],
  PlotRange → {{-2, 2}, {-2, 2}},
  ImageSize → 600
]
],
{i, Length[discData]}
];

Print["[4/12] Reversing frames for outward growth..."];
revFrames = Reverse[frames];

Print["[5/12] Slowing down animation (duplicating frames)..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[6/12] Rasterizing frames for neon glow stability..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[7/12] Exporting MP4 locally..."];
localFile = "gershgorin_temp.mp4";
Export[
  localFile,
  cleanFrames,
  "VideoEncoding" → "H264",
  "FrameRate" → 12
];

If[FileExistsQ[localFile],
  Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[]
];

Print["[8/12] Uploading to Wolfram Cloud..."];

(* ---- CORRECTED: Use CloudExport or CopyFile ---- *)

```

```

(* Method 1: Direct CloudExport (recommended) *)
cloudPath = ;
cloudObj = CloudExport[
  cleanFrames,
  "MP4",
  cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

(* Method 2: Alternative - Copy local file to cloud *)
(*
cloudObj = CloudObject[];
CopyFile[localFile, cloudObj];
SetPermissions[cloudObj, ];
*)

Print[];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"];
  Print["URL: ", cloudObj],
  Print["✗ Upload failed - trying alternative method..."];

(* Retry with explicit CloudPut *)
cloudObj = CloudPut[
  Import[localFile, "Video"],
  "jerryamyluv/VideoExports/Gershgorin_Brainanimation_v2.mp4"
];
SetPermissions[cloudObj, "Public"];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Retry successful!"];
  Print["URL: ", cloudObj],
  Print["✗ Both methods failed. Manual upload required."];
  Print["Local file saved at: ", localFile];
];
];

```

```
Print["[10/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share this URL: ", shareURL];
```

```
Print["[11/12] Displaying preview frame..."];
preview = First[frames];
preview
```

```
Print["[12/12] Complete! ✓"];
Print["Video location: ", cloudObj];
Print["Direct link: ", shareURL];
```

```
[1/12] Loading A-matrices...
```

```
[2/12] Computing Gershgorin discs...
```

```
[3/12] Building cyber-neon frames...
```

```
[4/12] Reversing frames for outward growth...
```

```
[5/12] Slowing down animation (duplicating frames)...
```

```
[6/12] Rasterizing frames for neon glow stability...
```

```
[7/12] Exporting MP4 locally...
```

```
✓ Local export successful
```

```
[8/12] Uploading to Wolfram Cloud...
```

```
[9/12] Verifying upload...
```

```
✓ Upload successful!
```

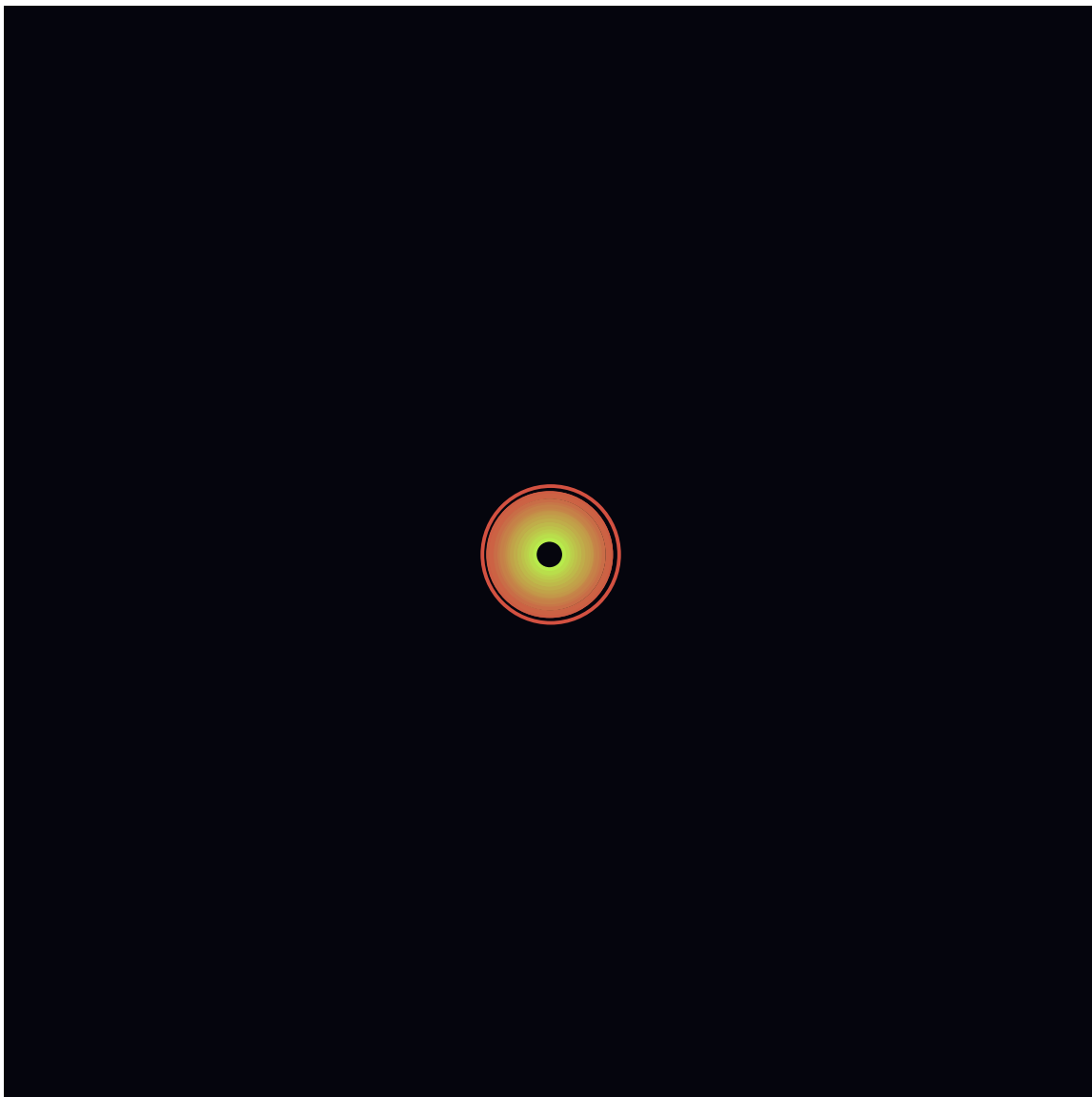
```
URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/]
      Gershgorin_Brainimation.mp4
```

```
[10/12] Getting shareable URL...
```

```
Share this URL: CloudObject[
  https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/][HTTPSPath]
  Gershgorin_Brainimation.mp4
```

```
[11/12] Displaying preview frame...
```

Out[382]=



[12/12] Complete! ✓

Video location:

CloudObject[[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/](https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/Gershgorin_Brainimation.mp4)
Gershgorin_Brainimation.mp4]

Direct link: CloudObject[

<https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/>[HTTPSPath]
Gershgorin_Brainimation.mp4]

In[386]:=

```
(* ===== *)
(* Script 3: Stability Tube Brainimation (Cyber-Neon Edition) *)
(* Final Version: Status + SlowMo + Stable Upload + Preview *)
(* ===== *)
```

```
ClearAll[];
```

```
Print["[1/12] Loading A-matrices..."];
```

```
(* ---- 0. Load A-matrices ---- *)
```

```
base = ;
```

```
files = {
```

```
  "Epoch1_A_Matrix.csv",
```

```
  "Epoch2_A_Matrix.csv",
```

```
  "Epoch3_A_Matrix.csv",
```

```
  "Epoch4_A_Matrix.csv",
```

```
  "Epoch5_A_Matrix.csv"
```

```
};
```

```
urls = base <> # & /@ files;
```

```
safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
```

```
  If[MatrixQ[raw], raw, $Failed]
```

```
];
```

```
Adata = Select[safeImport /@ urls, # != $Failed &];
```

```
Print["[2/12] Computing stability metrics..."];
```

```
(* ---- 1. Compute stability metrics ---- *)
```

```
eigvals = Eigenvalues /@ Adata;
```

```
spectralRadius = Max[Abs[#]] & /@ eigvals;
```

```
maxRealPart = Max[Re[#]] & /@ eigvals;
```

```
gershBound = (Max[Abs[Diagonal[#]] + (Total[Abs[#], {2}] - Abs[Diagonal[#]])) & /@ Adata;
```

```
epochs = Range[Length[Adata]];
```

```
(* Normalize for neon color scaling *)
```

```
allVals = Flatten[{spectralRadius, maxRealPart, gershBound}];
```

```
minV = Min[allVals];
```

```
maxV = Max[allVals];
```

```
neonColor[v_] := ColorData["NeonColors"][Rescale[v, {minV, maxV}]];
```



```

Print["[3/12] Building 3D neon tube frames..."];

(* ---- 2. Build frames ---- *)
frames = Table[
  Module[{e = epochs[[i]],
    Graphics3D[
      {
        (* Spectral radius tube *)
        {neonColor[spectralRadius[[i]],
          Thick,
          Tube[{e, spectralRadius[[i]], 0.0}, {e + 0.1, spectralRadius[[i]], 0.0}], 0.05]
        },

        (* Max real-part tube *)
        {neonColor[maxRealPart[[i]],
          Thick,
          Tube[{e, maxRealPart[[i]], 0.2}, {e + 0.1, maxRealPart[[i]], 0.2}], 0.05]
        },

        (* Gershgorin bound tube *)
        {neonColor[gershBound[[i]],
          Thick,
          Tube[{e, gershBound[[i]], -0.2}, {e + 0.1, gershBound[[i]], -0.2}], 0.05]
        }
      ],
    Background → RGBColor[0.02, 0.02, 0.05],
    PlotRange → {{0, Length[Adata] + 1}, {minV - 0.5, maxV + 0.5}, {-1, 1}},
    ImageSize → 700,
    Boxed → False,
    Lighting →
  ]
],
{i, Length[Adata]}
];

Print["[4/12] Reversing frames for outward growth..."];
revFrames = Reverse[frames];

Print["[5/12] Slowing down animation (duplicating frames)..."];
slowFactor = 3;

```

```

slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[6/12] Rasterizing frames for neon glow stability..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[7/12] Exporting MP4 locally..."];
localFile = "stability_tubes_temp.mp4";

Export[
  localFile,
  cleanFrames,
  "VideoEncoding" → "H264",
  "FrameRate" → 12
];

If[FileExistsQ[localFile],
  Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[]
];

Print["[8/12] Uploading to Wolfram Cloud..."];

(* ---- Primary upload method ---- *)
cloudPath = ;
cloudObj = CloudExport[
  cleanFrames,
  "MP4",
  cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[9/12] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"];
  Print["URL: ", cloudObj],

  Print["✗ Upload failed - trying fallback method..."];

  (* ---- Fallback: CloudPut local MP4 ---- *)

```

```

cloudObj = CloudPut[
  Import[localFile, "Video"],
  "jerryamyluv/VideoExports/StabilityTubes_Brainimation_v2.mp4"
];
SetPermissions[cloudObj, "Public"];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Fallback upload successful!"];
  Print["URL: ", cloudObj],
  Print["✗ Both methods failed. Manual upload required."];
  Print["Local file saved at: ", localFile];
];
];

Print["[10/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share this URL: ", shareURL];

Print["[11/12] Displaying preview frame..."];
preview = First[frames];
preview

Print["[12/12] Script 3 complete! ✓"];
Print["Video location: ", cloudObj];
Print["Direct link: ", shareURL];

[1/12] Loading A-matrices...
[2/12] Computing stability metrics...
[3/12] Building 3D neon tube frames...
[4/12] Reversing frames for outward growth...
[5/12] Slowing down animation (duplicating frames)...
[6/12] Rasterizing frames for neon glow stability...
[7/12] Exporting MP4 locally...
✓ Local export successful
[8/12] Uploading to Wolfram Cloud...
[9/12] Verifying upload...
✓ Upload successful!

```

URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/StabilityTubes_Brainimation.mp4]

[10/12] Getting shareable URL...

Share this URL: CloudObject[
https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/StabilityTubes_Brainimation.mp4][HTTPSPath]

[11/12] Displaying preview frame...

Out[426]=



[12/12] Script 3 complete! ✓

Video location:

CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/StabilityTubes_Brainimation.mp4]

Direct link: CloudObject[
https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/StabilityTubes_Brainimation.mp4][HTTPSPath]

In[430]:=

```
(* ===== *)
(* Script 4: 3D Gershgorin Shell Brainimation (Cyber-Neon) *)
(* ===== *)
```

```

ClearAll[];
Print["[1/12] Loading A-matrices..."];

base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];
Adata = Select[safeImport /@ urls, # != $Failed &];

Print["[2/12] Computing Gershgorin discs..."];

gershgorinDiscs[m_] := Module[{diag, radii},
  diag = Diagonal[m];
  radii = Total[Abs[m], {2}] - Abs[diag];
  Transpose[{diag, radii}]
];
discData = gershgorinDiscs /@ Adata;

allRadii = Flatten[discData[[All, All, 2]]];
minR = Min[allRadii]; maxR = Max[allRadii];
neonColor[r_] := ColorData["NeonColors"][Rescale[r, {minR, maxR}]];

Print["[3/12] Building 3D shell frames..."];

frames = Table[
  Module[{centers = discData[[i, All, 1]], radii = discData[[i, All, 2]]},
    Graphics3D[
      Table[
        {
          Directive[neonColor[radii[[k]]], Opacity[0.25]],
          Sphere[{Re[centers[[k]]], Im[centers[[k]]], 0}, radii[[k]]]
        }
      ]
    ]
  ]
];

```

```

    },
    {k, Length[centers]}
  ],
  Background → RGBColor[0.02, 0.02, 0.05],
  PlotRange → {{-2, 2}, {-2, 2}, {-2, 2}},
  Boxed → False,
  Lighting → "Neutral",
  ImageSize → 700
]
],
{i, Length[discData]}
];

Print["[4/12] Reversing frames..."];
revFrames = Reverse[frames];

Print["[5/12] Slowing down..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[6/12] Rasterizing..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[7/12] Exporting MP4 locally..."];
localFile = "gersh_shells_temp.mp4";
Export[localFile, cleanFrames, "VideoEncoding" → "H264", "FrameRate" → 12];

If[FileExistsQ[localFile], Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[]];

Print["[8/12] Uploading to Wolfram Cloud..."];

cloudPath = "jerryamyluv/VideoExports/GershgorinShells_Brainimation.mp4";
cloudObj = CloudExport[
  cleanFrames, "MP4", cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[9/12] Verifying upload..."];

```

```

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"]; Print["URL: ", cloudObj],
  Print["✗ Upload failed - trying fallback..."];
  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/GershgorinShells_Brainimation_v2.mp4"
  ];
  SetPermissions[cloudObj, "Public"];
  If[
    CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
    Print["✓ Fallback upload successful!"]; Print["URL: ", cloudObj],
    Print["✗ Both methods failed. Local file: ", localFile];
  ];
];

Print["[10/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj][HTTPSPath];
Print["Share URL: ", shareURL];

Print["[11/12] Preview frame..."];
preview = First[frames]; preview

Print["[12/12] Script 4 complete."];
Print["Video: ", cloudObj]; Print["Direct: ", shareURL];

[1/12] Loading A-matrices...
[2/12] Computing Gershgorin discs...
[3/12] Building 3D shell frames...
[4/12] Reversing frames...
[5/12] Slowing down...
[6/12] Rasterizing...
[7/12] Exporting MP4 locally...
✓ Local export successful
[8/12] Uploading to Wolfram Cloud...
[9/12] Verifying upload...
✓ Upload successful!
URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/
      GershgorinShells_Brainimation.mp4

```

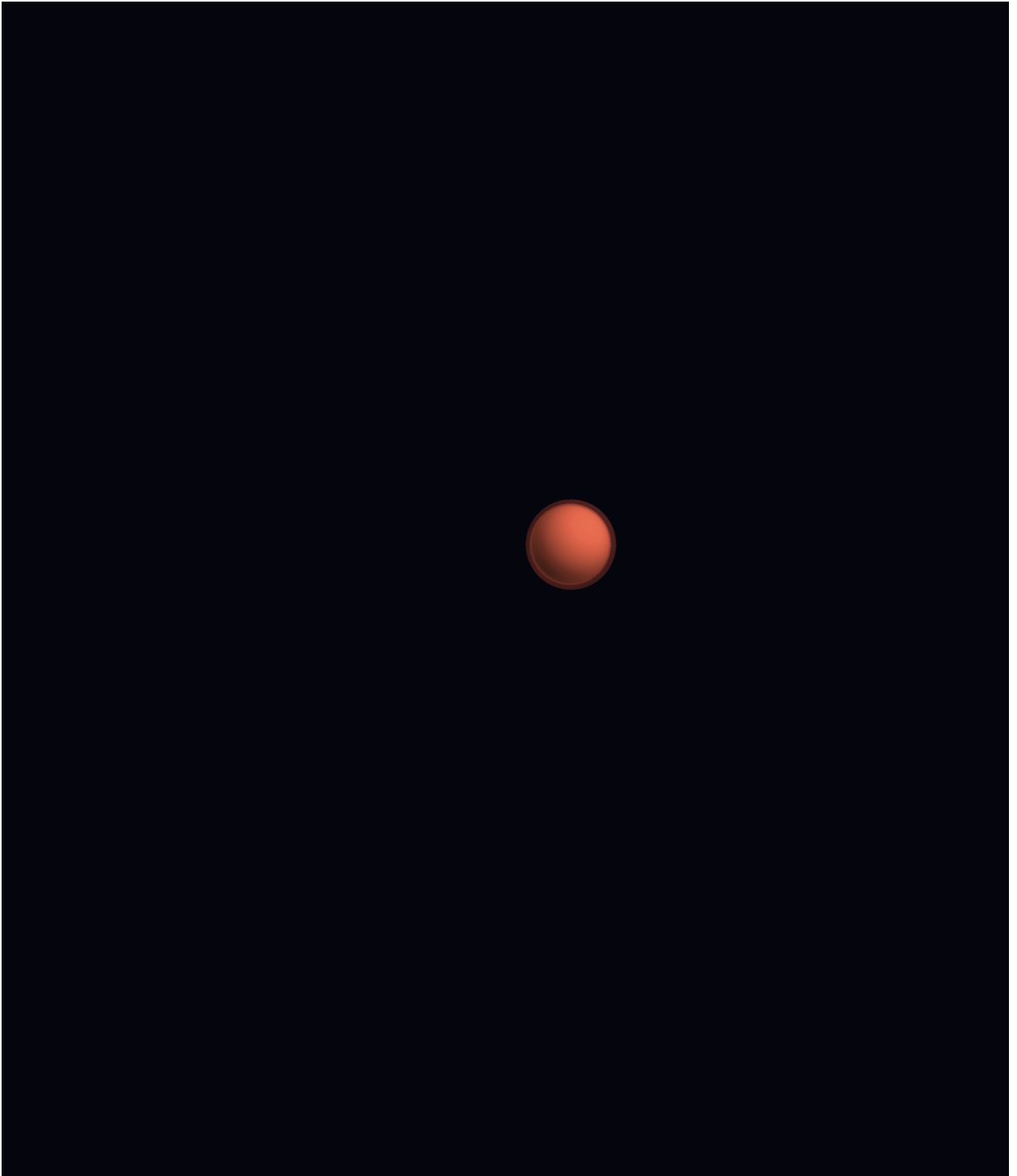
[10/12] Getting shareable URL...

Share URL: CloudObject[

[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/](https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/GershgorinShells_Brainimation.mp4)[HTTPSPath]
GershgorinShells_Brainimation.mp4

[11/12] Preview frame...

Out[465]=



```

[12/12] Script 4 complete.

Video: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/
      GershgorinShells_Brainimation.mp4

Direct: CloudObject[
      https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/][HTTPSPath]
      GershgorinShells_Brainimation.mp4

In[1]:= (* ===== *)
(* Script 5: Spectral Drift Vector Field Brainimation *)
(* ===== *)

ClearAll[];
Print["[1/12] Loading A-matrices..."];

base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];
Adata = Select[safeImport /@ urls, # != $Failed &];

Print["[2/12] Computing eigenvalues and drift..."];

eigvals = Eigenvalues /@ Adata;
drift = Table[
  If[i == 1, ConstantArray[0, Length[eigvals[[i]]],
    eigvals[[i]] - eigvals[[i - 1]]
  ],
  {i, Length[eigvals]}
];

allDriftMag = Abs[Flatten[drift]];
minD = Min[allDriftMag]; maxD = Max[allDriftMag];
neonColor[d_] := ColorData["NeonColors"][Rescale[d, {minD, maxD}]];

```

```

Print["[3/12] Building vector field frames..."];

frames = Table[
  Module[{vals = eigvals[[i]], dv = drift[[i]],
    Graphics[
      {
        Black,
        Arrowheads[0.02],
        Table[
          {
            neonColor[Abs[dv[[k]]],
            Arrow[{Re[vals[[k]], Im[vals[[k]]],
              {Re[vals[[k]], Im[vals[[k]]] + 0.5 {Re[dv[[k]], Im[dv[[k]]}}]}],
          },
          {k, Length[vals]}
        ]
      },
      Background → RGBColor[0.02, 0.02, 0.05],
      PlotRange → {{-2, 2}, {-2, 2}},
      ImageSize → 600
    ]
  ],
  {i, Length[eigvals]}
];

Print["[4/12] Reversing frames..."];
revFrames = Reverse[frames];

Print["[5/12] Slowing down..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[6/12] Rasterizing..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[7/12] Exporting MP4 locally..."];
localFile = "spectral_drift_temp.mp4";
Export[localFile, cleanFrames, "VideoEncoding" → "H264", "FrameRate" → 12];

```

```

If[FileExistsQ[localFile], Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[]];

Print["[8/12] Uploading to Wolfram Cloud..."];

cloudPath = "jerryamyluv/VideoExports/SpectralDrift_Brainimation.mp4";
cloudObj = CloudExport[
  cleanFrames, "MP4", cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[9/12] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"]; Print["URL: ", cloudObj],
  Print["✗ Upload failed - trying fallback..."];
  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/SpectralDrift_Brainimation_v2.mp4"
  ];
  SetPermissions[cloudObj, "Public"];
  If[
    CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
    Print["✓ Fallback upload successful!"]; Print["URL: ", cloudObj],
    Print["✗ Both methods failed. Local file: ", localFile];
  ];
];

Print["[10/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share URL: ", shareURL];

Print["[11/12] Preview frame..."];
preview = First[frames]; preview

Print["[12/12] Script 5 complete."];
Print["Video: ", cloudObj]; Print["Direct: ", shareURL];

```

```

[1/12] Loading A-matrices...
[2/12] Computing eigenvalues and drift...
[3/12] Building vector field frames...
[4/12] Reversing frames...
[5/12] Slowing down...
[6/12] Rasterizing...
[7/12] Exporting MP4 locally...
✓ Local export successful
[8/12] Uploading to Wolfram Cloud...
[9/12] Verifying upload...
✓ Upload successful!
URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/
      SpectralDrift_Brainimation.mp4
[10/12] Getting shareable URL...
Share URL: CloudObject[
      https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/][HTTPSPath]
      SpectralDrift_Brainimation.mp4
[11/12] Preview frame...

```

Out[36]=



[12/12] Script 5 complete.

Video: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/SpectralDrift_Brainimation.mp4]

Direct: CloudObject[
https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/SpectralDrift_Brainimation.mp4][HTTPSPath]

```
In[38]:= (* ===== *)
(* Script 6: A-Matrix Heatmap Evolution Brainimation *)
(* ===== *)
```

```
ClearAll[];
```

```
Print["[1/12] Loading A-matrices..."];
```

```

base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];
Adata = Select[safeImport /@ urls, # != $Failed &];

Print["[2/12] Building heatmap frames..."];

allVals = Flatten[Adata];
minV = Min[allVals]; maxV = Max[allVals];

frames = Table[
  MatrixPlot[
    Adata[[i]],
    ColorFunction -> (ColorData["NeonColors"][Rescale[#, {minV, maxV}]] &),
    ColorFunctionScaling -> False,
    PlotRange -> {minV, maxV},
    Frame -> False,
    Background -> Black,
    ImageSize -> 600
  ],
  {i, Length[Adata]}
];

Print["[3/12] Reversing frames..."];
revFrames = Reverse[frames];

Print["[4/12] Slowing down..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[5/12] Rasterizing..."];

```

```

cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[6/12] Exporting MP4 locally..."];
localFile = "A_matrix_heatmap_temp.mp4";
Export[localFile, cleanFrames, "VideoEncoding" → "H264", "FrameRate" → 12];

If[FileExistsQ[localFile], Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[]];

Print["[7/12] Uploading to Wolfram Cloud..."];

cloudPath = "jerryamyluv/VideoExports/A_MatrixHeatmap_Brainimation.mp4";
cloudObj = CloudExport[
  cleanFrames, "MP4", cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[8/12] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"]; Print["URL: ", cloudObj],
  Print["✗ Upload failed - trying fallback..."];
  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/A_MatrixHeatmap_Brainimation_v2.mp4"
  ];
  SetPermissions[cloudObj, "Public"];
  If[
    CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
    Print["✓ Fallback upload successful!"]; Print["URL: ", cloudObj],
    Print["✗ Both methods failed. Local file: ", localFile];
  ];
];

Print["[9/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share URL: ", shareURL];

```



```
Print["[10/12] Preview frame..."];
preview = First[frames]; preview
```

```
Print["[11/12] Script 6 complete."];
Print["Video: ", cloudObj]; Print["Direct: ", shareURL];
```

```
[1/12] Loading A-matrices...
```

```
[2/12] Building heatmap frames...
```

```
[3/12] Reversing frames...
```

```
[4/12] Slowing down...
```

```
[5/12] Rasterizing...
```

```
[6/12] Exporting MP4 locally...
```

```
✓ Local export successful
```

```
[7/12] Uploading to Wolfram Cloud...
```

```
[8/12] Verifying upload...
```

```
✓ Upload successful!
```

```
URL: CloudObject[
```

```
  https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/A_MatrixHeatmap[
    _Brainimation.mp4
```

```
[9/12] Getting shareable URL...
```

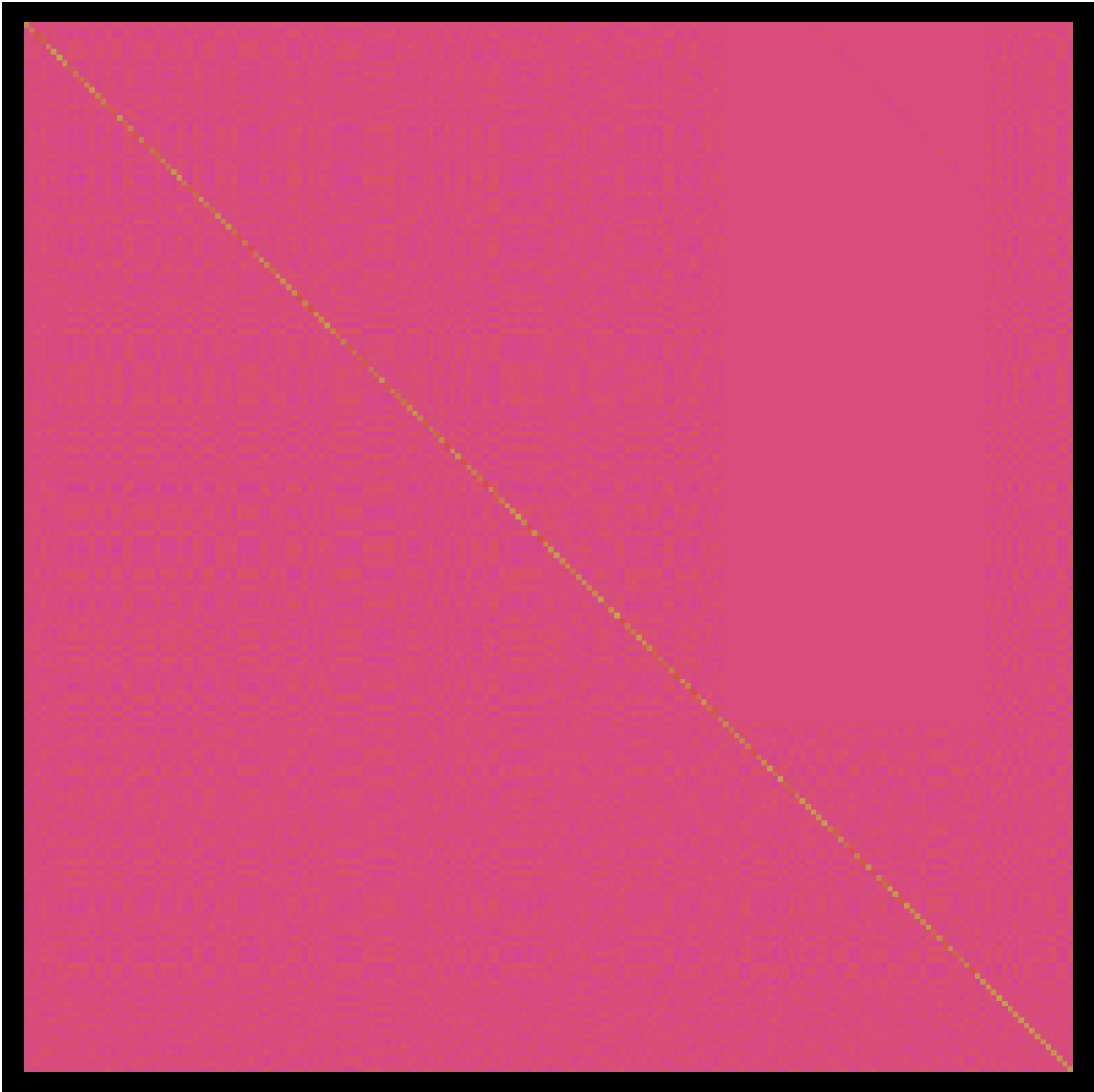
```
Share URL: CloudObject[
```

```
  https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/A_MatrixHeatmap[
    _Brainimation.mp4
```

```
  HTTPSPATH]
```

```
[10/12] Preview frame...
```

Out[69]=



[11/12] Script 6 complete.

Video: CloudObject[

https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/A_MatrixHeatmap_Brainanimation.mp4

Direct: CloudObject[

https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/A_MatrixHeatmap_Brainanimation.mp4

HTTPSPATH]

```
In[71]:= (* ===== *)
(* Script 7: Logic Map / A-Matrix Structure Brainanimation *)
(* ===== *)
```

```

ClearAll[];
Print["[1/12] Loading A-matrices..."];

base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
};
urls = base <> # & /@ files;

safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
  If[MatrixQ[raw], raw, $Failed]
];
Adata = Select[safeImport /@ urls, # != $Failed &];

Print["[2/12] Building logic-map frames..."];

(* Threshold + sign map: -1, 0, +1 *)
threshold = 0.01;

logicMap[m_] := Sign[Clip[m, {-threshold, threshold}]] /. {
  -1 → -1, 0 → 0, 1 → 1
};

frames = Table[
  ArrayPlot[
    logicMap[Adata[[i]]],
    ColorRules → {
      -1 → ColorData["NeonColors"][0.1],
      0 → Black,
      1 → ColorData["NeonColors"][0.9]
    },
    Frame → False,
    Background → Black,
    ImageSize → 600
  ],
  {i, Length[Adata]}
];

```

```

Print["[3/12] Reversing frames..."];
revFrames = Reverse[frames];

Print["[4/12] Slowing down..."];
slowFactor = 3;
slowFrames = Flatten[Table[revFrames[[i]], {i, Length[revFrames]}, slowFactor]];

Print["[5/12] Rasterizing..."];
cleanFrames = Rasterize[#, "Image"] & /@ slowFrames;

Print["[6/12] Exporting MP4 locally..."];
localFile = "logic_map_temp.mp4";
Export[localFile, cleanFrames, "VideoEncoding" → "H264", "FrameRate" → 12];

If[FileExistsQ[localFile], Print["✓ Local export successful"],
  Print["✗ Local export FAILED"]; Abort[]];

Print["[7/12] Uploading to Wolfram Cloud..."];

cloudPath = "jerryamyluv/VideoExports/LogicMap_Brainimation.mp4";
cloudObj = CloudExport[
  cleanFrames, "MP4", cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[8/12] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"]; Print["URL: ", cloudObj],
  Print["✗ Upload failed - trying fallback..."];
  cloudObj = CloudPut[
    Import[localFile, "Video"],
    "jerryamyluv/VideoExports/LogicMap_Brainimation_v2.mp4"
  ];
  SetPermissions[cloudObj, "Public"];
  If[
    CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
    Print["✓ Fallback upload successful!"]; Print["URL: ", cloudObj],
    Print["✗ Both methods failed. Local file: ", localFile];
  ]

```

```
];
];
```

```
Print["[9/12] Getting shareable URL..."];
shareURL = CloudObject[cloudObj]["HTTPSPath"];
Print["Share URL: ", shareURL];
```

```
Print["[10/12] Preview frame..."];
preview = First[frames]; preview
```

```
Print["[11/12] Script 7 complete."];
Print["Video: ", cloudObj]; Print["Direct: ", shareURL];
```

```
[1/12] Loading A-matrices...
```

```
[2/12] Building logic-map frames...
```

```
[3/12] Reversing frames...
```

```
[4/12] Slowing down...
```

```
[5/12] Rasterizing...
```

```
[6/12] Exporting MP4 locally...
```

```
✓ Local export successful
```

```
[7/12] Uploading to Wolfram Cloud...
```

```
[8/12] Verifying upload...
```

```
✓ Upload successful!
```

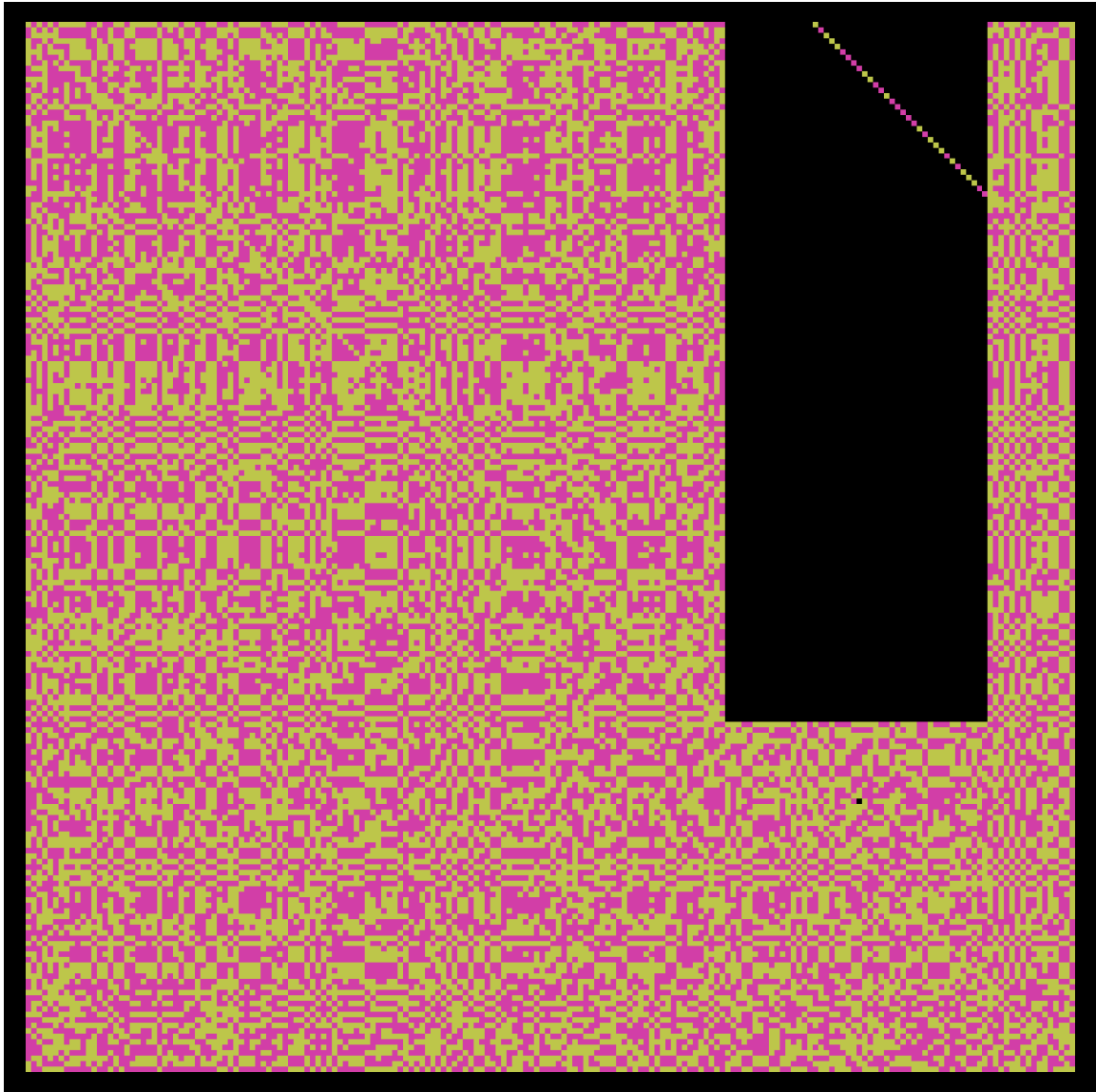
```
URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/
                LogicMap_Brainimation.mp4
```

```
[9/12] Getting shareable URL...
```

```
Share URL: CloudObject[
    https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/]HTTPSPath
    LogicMap_Brainimation.mp4
```

```
[10/12] Preview frame...
```

Out[102]=



[11/12] Script 7 complete.

Video: CloudObject[[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/](https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/LogicMap_Brainimation.mp4)
LogicMap_Brainimation.mp4

Direct: CloudObject[
[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/](https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/LogicMap_Brainimation.mp4)][HTTPSPath]
LogicMap_Brainimation.mp4

```
In[1]:= (* ===== *)
(* Script 8: OPTIMIZED for Wolfram Cloud Memory *)
(* ===== *)
```

ClearAll[];

```
logicShellMode = 1;
```

```
Print["[1/18] Loading A-matrices..."];
```

```
base = "https://www.wolframcloud.com/obj/jerryamyluv/A_matrices";
```

```
files = {
  "Epoch1_A_Matrix.csv",
  "Epoch2_A_Matrix.csv",
  "Epoch3_A_Matrix.csv",
  "Epoch4_A_Matrix.csv",
  "Epoch5_A_Matrix.csv"
```

```
};
```

```
urls = base <> # & /@ files;
```

```
safeImport[url_] := Module[{raw = Quiet@Import[url, "CSV"]},
```

```
  If[MatrixQ[raw], raw, $Failed]
```

```
];
```

```
Adata = Select[safeImport /@ urls, # != $Failed &];
```

```
If[Adata == {}, Print["x No valid A-matrices. Aborting."]; Abort[]];
```

```
Print["[2/18] Computing stability metrics..."];
```

```
analyzeA[matrix_] := Module[
```

```
{A = matrix, eigs, vecs, diag, rowSums, radii, rho, gersh},
```

```
eigs = Eigenvalues[A];
```

```
vecs = Eigenvectors[A];
```

```
diag = Diagonal[A];
```

```
rowSums = Total[Abs[#]] & /@ A;
```

```
radii = rowSums - Abs[diag];
```

```
rho = Max[Abs[eigs]];
```

```
gersh = Max[diag + radii];
```

```
<|"Matrix" → A, "Eigenvalues" → eigs, "Eigenvectors" → vecs,
```

```
"Diag" → diag, "Radii" → radii, "SpectralRadius" → rho,
```

```
"GershgorinBound" → gersh>
```

```
];
```

```

results = analyzeA /@ Adata;
epochNums = Range[Length[results]];

rhoAll = results[[All, "SpectralRadius"]];
gershAll = results[[All, "GershgorinBound"]];

minRho = Min[rhoAll]; maxRho = Max[rhoAll];
minG = Min[gershAll]; maxG = Max[gershAll];

Print["[3/18] Defining neon color maps..."];

neonEigen[v_] := ColorData["NeonColors"][Rescale[v, {minRho, maxRho}]];
neonShell[v_] := ColorData["NeonColors"][Rescale[v, {minG, maxG}]];
neonFiber[v_] := ColorData["NeonColors"][Rescale[v, {0, 1}]];
neonTensor[v_] := ColorData["NeonColors"][Rescale[v, {0, 1}]];
neonTube[v_] := ColorData["NeonColors"][Rescale[v, {minRho, maxRho}]];

Print["[4/18] Computing composite geometry..."];

(* OPTIMIZED: Reduced complexity *)
computeComposite[res_, epoch_] := Module[
  {A, vals, vecs, diag, radii, pts, centerShift,
   spheres, fiberHalo, tensorRows, tensorArrows, rho, gersh,
   n, reVals, imVals},

  A = res[];
  vals = res[];
  vecs = res[];
  diag = res[];
  radii = res[];
  rho = res[];
  gersh = res[];

  (* Limit to first 50 eigenvalues for speed *)
  n = Min[Length[vals], Length[radii], 50];
  vals = Take[vals, n];
  vecs = Take[vecs, n];
  radii = Take[radii, n];

  reVals = Re[vals];
  imVals = Im[vals];

```



```

(* Create points *)
pts = Table[{reVals[[i]], imVals[[i]], radii[[i]]}, {i, n}];

(* Center the constellation - CORRECT FORM *)
centerShift = Mean[pts];
pts = (# - centerShift) & /@ pts; (* Map form to avoid Thread error *)

spheres = Table[{pts[[i]], radii[[i]]}, {i, Length[pts]}];

(* Simplified fiber halo - fewer fibers per point *)
fiberHalo = Table[
Module[
{
p = pts[[i]],
vec = vecs[[i]],
vec3, vec3padded, dir
},

vec3 = Take[vec, UpTo[3]];
vec3padded = PadRight[vec3, 3, 0];

dir = If[Norm[vec3padded] == 0,
{0, 0, 1},
Normalize[Re[vec3padded]]
];

(* Only 4 fibers instead of 12 *)
Table[
Arrow[{p, p + 0.15 Normalize[dir + 0.2 {Cos[t], Sin[t], 0}]}],
{t, 0, 2 Pi, Pi/2}
],
{i, Length[pts]}
] // Flatten;

(* Reduced tensor arrows *)
tensorRows = A[All, 1 ;; Min[3, Dimensions[A][[2]]]];
tensorArrows = Table[
With[{
base = {i, -1.5, 0},
dir = Normalize[

```

```

    PadRight[tensorRows[[i]], 3, 0] /. {0., 0., 0.} -> {0., 0., 1.}
  ]
},
  Arrow[{base, base + 0.3 dir}]
],
{i, Min[Length[tensorRows], 20]} (* Reduced from 40 *)
];

<| -> pts, -> spheres, -> fiberHalo,
  -> tensorArrows, -> rho, -> gersh,
  -> epoch|>
];

composites = MapThread[computeComposite, {results, epochNums}];

Print["[5/18] Building stability tubes..."];

stabilityTubes[epoch_, rho_, gersh_] := Module[
  {e = epoch},
  {
    Tube[{e, 2.0, 0},
      {e + 0.1, 2.0 + 0.2 Rescale[rho, {minRho, maxRho}], 0}], 0.05],
    Tube[{e, 2.0, -0.5},
      {e + 0.1, 2.0 + 0.2 Rescale[gersh, {minG, maxG}], -0.5}], 0.05]
  }
];

tubesPerEpoch = Map[
  stabilityTubes[#, "Epoch"], #["Rho"], #["Gersh"]] &,
  composites
];

Print["[6/18] Preparing logic-shell geometry..."];

threshold = 0.01;
logicMaps = Sign[Clip[#, {-threshold, threshold}]] & /@ (results[[All, "Matrix"]]);
logicShellRadius = 3.0;

(* OPTIMIZED: Drastically reduced resolution *)
logicShellPoints[logic_, mode_] := Module[

```

```

{n, coords, vals, samplingFactor},
n = Length[logic];
samplingFactor = Max[1, Floor[n/20]]; (* Sample every 20th element *)

vals = Flatten[logic];
coords = Flatten[
  Table[
    Module[{ $\theta = i/n \text{ Pi}$ ,  $\varphi = j/n^2 \text{ Pi}$ ,
      r = logicShellRadius},
      r {Sin[ $\theta$ ] Cos[ $\varphi$ ], Sin[ $\theta$ ] Sin[ $\varphi$ ], Cos[ $\theta$ ]}
    ],
    {i, 1, n, samplingFactor}, {j, 1, n, samplingFactor}
  ],
  1
];

(* Take corresponding values *)
vals = Take[vals, Length[coords]];

{coords, vals}
];

Print["[7/18] Building 3D frames (OPTIMIZED)..."];

frames = Table[
  Module[
    {
      c = composites[[i]],
      tubes = tubesPerEpoch[[i]],
      pts, spheres, fibers, tensorArrows, rho,
      logic = logicMaps[[i]],
      coords, vals, logicGraphics,
      hueShift
    },
    {
      pts = c["Points"];
      spheres = c["Spheres"];
      fibers = c["Fibers"];
      tensorArrows = c["TensorArrows"];
      rho = c["Rho"];

      hueShift = Mod[0.1*i, 1];
    }
  ]
];

```

```

{coords, vals} = logicShellPoints[logic, logicShellMode];

logicColor[val_] := Which[
  val == 1, Blend[{neonShell[1], ColorData["NeonColors"][hueShift]}, 0.35],
  val == -1, Blend[{neonShell[0], ColorData["NeonColors"][hueShift]}, 0.35],
  True, Opacity[0]
];

(* Simplified logic graphics - only mode 1 for now *)
logicGraphics = Table[
  {Opacity[0.12], logicColor[vals[[k]]], PointSize[0.015], Point[coords[[k]]],
  {k, Length[coords]}
];

Graphics3D[
{
  logicGraphics,

  Table[{Opacity[0.10], neonShell[spheres[[k, 2]]],
    Sphere[spheres[[k, 1]], spheres[[k, 2]]],
    {k, Length[spheres]}],

  Table[{Opacity[0.30], neonEigen[rho], PointSize[0.02],
    Point[pts[[k]]],
    {k, Length[pts]}],

  Table[{Opacity[0.45], neonFiber[0.7], Thick, fibers[[k]]},
    {k, Length[fibers]}],

  Table[{Opacity[0.20], neonTensor[0.3], tensorArrows[[k]]},
    {k, Length[tensorArrows]}],

  {Opacity[0.20], neonTube[rho], tubes[[1]], tubes[[2]]}
},
Background → RGBColor[0.02, 0.02, 0.05],
Boxed → False,
Lighting → "Neutral",
PlotRange → All,
ImageSize → 400 (* REDUCED from 700 *)
]

```

```

    ],
    {i, Length[composites]}
  ];

Print["[8/18] Reversing frames..."];
revFrames = Reverse[frames];

Print["[9/18] Skipping interpolation for speed..."];
(* Skip interpolation to reduce frame count *)
smoothFrames = revFrames;

Print["[10/18] Slowing down animation..."];
slowFactor = 2;
slowFrames = Flatten[Table[smoothFrames[[i]], {i, Length[smoothFrames]}, slowFactor]];

Print["[11/18] Rasterizing frames WITH PROGRESS..."];
Print["Total frames to rasterize: ", Length[slowFrames]];

(* Rasterize with progress indicator *)
cleanFrames = Table[
  (
    If[Mod[i, 5] == 0, Print[, i, , Length[slowFrames]]];
    Rasterize[slowFrames[[i]], , ImageSize -> 400]
  ),
  {i, Length[slowFrames]}
];

Print["[12/18] Exporting MP4 locally..."];

localFile = "neural_organ_optimized.mp4";
Export[localFile, cleanFrames, "VideoEncoding" -> "H264", "FrameRate" -> 12];

If[FileExistsQ[localFile],
  Print["✓ Local export successful: ", localFile],
  Print["✗ Local export FAILED"]; Abort[]
];

Print["[13/18] Uploading to Wolfram Cloud..."];

cloudPath = "jerryamyluv/VideoExports/NeuralOrgan_Optimized.mp4";

```

```

cloudObj = CloudExport[
  cleanFrames, "MP4", cloudPath,
  "FrameRate" → 12,
  Permissions → "Public"
];

Print["[14/18] Verifying upload..."];

If[
  CloudObjectInformation[cloudObj, "LastModified"] != Missing["NotAvailable"],
  Print["✓ Upload successful!"];
  Print["URL: ", cloudObj],
  Print["✗ Upload failed"];
];

Print["✓ Script complete!"];
CloudObject[cloudObj][["HTTPSPath"]
[1/18] Loading A-matrices...
[2/18] Computing stability metrics...
[3/18] Defining neon color maps...
[4/18] Computing composite geometry...
[5/18] Building stability tubes...
[6/18] Preparing logic-shell geometry...
[7/18] Building 3D frames (OPTIMIZED)...
[8/18] Reversing frames...
[9/18] Skipping interpolation for speed...
[10/18] Slowing down animation...
[11/18] Rasterizing frames WITH PROGRESS...
Total frames to rasterize: 10
  Rasterizing frame 5/10
  Rasterizing frame 10/10
[12/18] Exporting MP4 locally...
✓ Local export successful: neural_organ_optimized.mp4
[13/18] Uploading to Wolfram Cloud...
[14/18] Verifying upload...
✓ Upload successful!
URL: CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/]
      NeuralOrgan_Optimized.mp4

```

✓ Script complete!

Out[57]=

```
CloudObject[https://www.wolframcloud.com/obj/jerryamyluv/jerryamyluv/VideoExports/[  
    NeuralOrgan_Optimized.mp4  
    HTTPSPath]
```